Real-Time Seam Path Detection for Decorative Stitch Adjustment on Fabrics Using Image Processing

Echtzeit-Nahtpfaderkennung zur Zierstichanpassung auf Textilien mittels Bildverarbeitung

Sara Bavi

Advisor

Prof. Dr.-Ing. Daniel Görges

Institute of Electromobility Department of Electrical and Computer Engineering

Rheinland-Pfälzische Technische Universität Kaiserslautern Landau

11. April 2026

Contents

List of Figures III

List of Tables V

Nomenclature VII

Abstract 1

1. Introduction 3
1.1. Motivation . . . 3
1.2. Problem Statement . . . 3
1.3. State of the Art . . . 4
1.4. Objectives . . . 4
1.5. Contributions . . . 5
1.6. System Overview . . . 5
1.7. Challenges . . . 6
1.8. Outline . . . 6

2. System Concept and Hardware 7
2.1. Hardware Components . . . 7
2.2. Control Concepts and Dynamic Programming . . . 9
2.2.1. Implementation-Level Optimization . . . 10
2.2.2. Bidirectional Seam Estimation . . . 12
2.2.3. Polynomial Trajectory Approximation . . . 14
2.3. Development Methodology . . . 17

3. Implementation 19
3.1. Image Processing Method . . . 19
3.1.1. Image Acquisition . . . 19
3.1.2. CLAHE-Based Contrast Enhancement . . . 20
3.1.3. Gaussian Smoothing . . . 26
3.1.4. Gabor Filtering for Texture Enhancement . . . 30
3.1.5. Summary of Preprocessing Effects . . . 35
3.2. Seam Path Detection and Fitting . . . 36
3.2.1. Dynamic Programming . . . 36
3.2.2. Bidirectional Seam Detection . . . 41
3.2.3. Polynomial Curve Fitting . . . 42
3.2.4. Distance Measurement . . . 42
3.2.5. Real-Time Implementation Using Threading . . . 43
3.2.6. Threaded Camera Acquisition . . . 47

3.2.7. Performance Considerations 48
3.3. Result after System Integration 48
3.4. Software Output and Visualization 49
3.5. Final Automated Stitching Results 50

4. Verification and Validation **53**
4.1. System Calibration, Path Control and Visualization 53
4.1.1. Calibration and Metric Conversion 53
4.1.2. Multi-Point Deviation Evaluation 54
4.1.3. Control-Oriented Interpretation 54
4.1.4. Visualization and Real-Time Feedback 54
4.2. Challenges and Solutions 55
4.2.1. Discussion of System Robustness 57
4.2.2. Performance Trade-Offs 57

5. Conclusion and Future Work **59**
5.1. Conclusion 59
5.2. Future Work 60
5.2.1. Closed-Loop Control Integration 60
5.2.2. Robustness Enhancement and Noise Compensation 61
5.2.3. Algorithm Optimization for Industrial Deployment 61
5.2.4. Hardware Acceleration and Embedded Deployment 61
5.2.5. Extended Experimental Validation 62

A. Full Seam Detection Script **63**

B. Real-Time Threaded Full Seam Detection Script **69**

Bibliography **75**

List of Figures

1.1. Seam path detection pipeline 6

2.1. Hardware setup and physical scale. 7
2.2. Close-up view of the bare XIMEA camera module. 8
2.3. Internal image data flow and processing pipeline of the XIMEA camera via xiAPI [XIM17]. 8
2.4. Side view of the fabric positioned under the sewing machine mechanism. 9
2.5. Computational architecture of the optimized DP implementation. (A) The Numba JIT compilation pipeline, bypassing the slow Python interpreter to execute raw machine code. (B) The memory structure during execution, illustrating the parallel updating of the Cumulative Cost Matrix and the Predecessor Matrix, which allows for $\mathcal{O}(H)$ backtracking speed. 12
2.6. Conceptual visualization of the bidirectional seam estimation strategy. (A) The Forward DP pass is distracted by a dark artifact on the left, causing a path deviation. (B) The Reverse DP pass evaluated on the flipped domain avoids the left artifact but is distracted by a secondary right artifact. (C) Row-wise energy arbitration successfully fuses the two paths by dynamically selecting the minimum local cost, bypassing both artifacts to perfectly track the true physical seam. 14
2.7. Conceptual illustration of polynomial trajectory approximation on a noisy discrete path. Degree 1 provides a robust linear trend but fails to capture macro-curvature (underfitting). Degree 3 accurately captures the seam geometry while smoothing local pixel noise. Degree 15 demonstrates severe overfitting (Runge's phenomenon), erroneously following high-frequency noise and drastically destabilizing the control trajectory. 16
2.8. The V-Model methodology applied to the project development. 18

3.1. Conceptual overview of the image acquisition stage. (A) The raw sensor image is rotated 180° via an affine transformation to physically align the image matrix coordinate system (r, c) with the fabric feed direction. (B) The MONO8 grayscale acquisition maps physical brightness directly to an 8-bit integer matrix, naturally forming a cost field where the dark physical seam corresponds to localized mathematical minimums. 20
3.2. Comparison between low-contrast image, global histogram equalization, and local adaptive equalization (CLAHE). The corresponding histograms and cumulative distributions are shown below each image. CLAHE enhances local contrast while avoiding over-amplification of noise. 23

3.3. Effect of Gaussian smoothing analyzed in both spatial and frequency domains. The Gaussian filter successfully suppresses high-frequency noise and fine fabric textures (visible in the outer regions of the frequency spectra) while preserving the low-frequency global structure of the seam path. 30
3.4. Conceptual overview of the 2D Gabor filter. (A) Spatial domain parameter sweep demonstrating the effect of varying orientation (θ) and wavelength (λ). (B) 1D mathematical decomposition illustrating the multiplication of a sinusoidal carrier by a Gaussian envelope for spatial localization. (C) Conceptual 2D frequency response, contrasting the central low-pass behavior of a Gaussian filter with the directional, localized band-pass behavior of a Gabor filter. 33
3.5. Visual justification for the Gabor kernel spatial support. Setting the kernel width to 6σ safely captures the effective area of the Gaussian envelope without introducing truncation artifacts or unnecessary computational overhead. 34
3.6. Sequential visualization of the image processing pipeline demonstrating the isolation of the seam path. 35
3.7. Conceptual illustration of dynamic programming for seam extraction. The optimal seam is obtained by propagating cumulative costs row by row and then reconstructing the minimum-cost path by backtracking through the stored predecessors. 39
3.8. Mechanism and effect of the diagonal transition penalty in the DP algorithm. (A) Local state transition model demonstrating the artificial cost $+P$ applied to lateral movements. (B) Global tracking simulation illustrating how a purely greedy algorithm ($P = 0.0$) gets derailed by dark isolated noise, whereas the regularized algorithm ($P = 0.05$) maintains structural continuity. 41
3.9. The implemented multi-threaded software architecture. Decoupling the I/O-bound tasks (camera acquisition and disk saving) from the compute-bound image processing pipeline ensures that hardware latency does not disrupt the real-time continuous control of the sewing machine. 46
3.10. Results of the integrated real-time seam detection system, showing robust tracking across varying motion conditions and different material textures. 49
3.11. Command Prompt execution displaying the real-time processing frame rate and metric deviations. 50
3.12. Final physical results of the automated decorative stitching process, demonstrating highly precise and parallel alignment relative to the main seam. 51

5.1. Visual summary of the intelligent real-time seam tracking concept. . . . 59

List of Tables

4.1. Summary of Challenges and Implemented Solutions 56

Nomenclature

Symbol	Description
$I_0(r, c)$	Raw captured grayscale image
H	Image height in pixels
W	Image width in pixels
r, c	Row and column pixel coordinates
I_{clahe}	Image after Contrast Limited Adaptive Histogram Equalization (CLAHE)
I_{gauss}	Image after Gaussian smoothing
I_{gabor}	Image after Gabor filtering
$G_\sigma(x, y)$	Two-dimensional Gaussian kernel
σ	Standard deviation (Gaussian envelope/blur radius)
$g(x, y)$	Two-dimensional Gabor kernel
θ	Orientation of the Gabor filter
λ	Wavelength of the Gabor filter sinusoidal factor
γ	Spatial aspect ratio of the Gabor filter
ψ	Phase offset of the Gabor filter
$E(r, c)$	Local pixel energy / cost function
$C(r, c)$	Cumulative cost matrix for dynamic programming
P	Diagonal transition penalty for dynamic programming
x_{ref}	Horizontal coordinate of the vertical reference line
d_{pix}	Seam deviation distance measured in pixels
d_{mm}	Seam deviation distance measured in millimeters
s	Calibration factor converting pixels to millimeters
FPS	Frames per second

Abstract

This thesis focuses on the development of an intelligent system designed to detect the seam path on fabrics in real time. By leveraging image processing, optimal control, and dynamic programming, the system aims to automatically adjust decorative stitch positions according to existing swing lines. This automation eliminates the need for manual alignment, thereby significantly improving precision, uniformity, and production speed. The developed solution has practical applications in modern textile manufacturing, including upholstery and automotive seat production.

1. Introduction

1.1. Motivation

The increasing demand for high-quality textile products in industrial manufacturing has led to a growing need for automation in sewing processes. In particular, decorative sewing operations require high precision and consistency, which are difficult to achieve using traditional manual alignment methods. Manual positioning of fabrics and seam tracking is not only time-consuming but also prone to human error, resulting in inconsistencies in stitch placement and reduced product quality.

Automating seam path detection offers significant advantages, including improved production speed, enhanced repeatability, and reduced dependency on skilled operators. This is especially critical in industries such as automotive upholstery, furniture manufacturing, and garment production, where both aesthetic quality and efficiency are essential.

Therefore, the development of an intelligent system capable of accurately detecting seam paths in real time is of great importance for modern industrial sewing applications.

1.2. Problem Statement

Despite advances in industrial automation, reliable real-time detection of seam paths on fabrics remains a challenging task. Fabrics often exhibit complex textures, low contrast between seam and background, varying illumination conditions, and dynamic deformations during motion.

Furthermore, high-speed production environments require the system to process image data at high frame rates without missing critical features. Machine vibrations and motion blur introduce additional noise, making seam detection even more difficult.

Traditional image processing techniques often fail to provide stable and accurate seam localization under such conditions. Therefore, a robust and efficient solution is required that can:

- Accurately detect seam paths under varying lighting conditions,

- Handle different fabric textures and patterns,
- Detect both straight and curved seam geometries,
- Operate reliably in real time under high-speed conditions,
- Provide stable outputs suitable for machine control.

1.3. State of the Art

Modern industrial sewing machines, such as the VETRON machine utilized in this study, offer advanced mechanical capabilities and programmable stitching patterns. However, real-time adaptation based on visual feedback remains a complex challenge. Recent developments in computer vision have introduced various approaches for seam detection, including edge detection, gradient-based methods, and texture analysis [MKP09, PVW17, RZR+20, LLL+21]. More advanced techniques incorporate directional filters and frequency-based methods to enhance seam visibility [JRL96, GPK02, Kam14, CZGL20].

Despite these advancements, several limitations remain. Many existing approaches are sensitive to illumination changes and fabric variability [ZXW13]. Additionally, achieving both robustness and real-time performance simultaneously is still an open challenge [KCTY23]. To address these limitations, more sophisticated image processing pipelines are required, combining contrast enhancement [Sha23, VJDBSS22], feature extraction, and robust curve fitting techniques [BGH03].

1.4. Objectives

The primary objective of this thesis is to develop a robust and real-time capable seam path detection system for industrial sewing applications.

The system is designed to:

- Detect seam paths accurately using image processing techniques,
- Automatically adjust decorative stitch positions according to the detected seam,
- Handle a wide range of fabric materials, textures, and patterns,
- Ensure stable performance under varying environmental conditions,
- Extract key positional information required for machine control,

- Enable reliable communication with the machine control unit via the CAN protocol.

1.5. Contributions

This thesis presents the design and implementation of a complete seam path detection system. The main contributions are summarized as follows:

- Development of a real-time image processing pipeline for seam detection,
- Application of contrast enhancement techniques to improve seam visibility,
- Use of directional filtering methods for robust feature extraction,
- Implementation of a reliable curve fitting approach for seam path estimation,
- Extraction of control-relevant parameters for machine integration,
- Integration of the system with an industrial control unit using CAN communication,
- Experimental validation under real-world operating conditions.

1.6. System Overview

The proposed system follows a structured processing pipeline, starting from image acquisition and ending with control feedback to the sewing machine. The overall workflow is illustrated in Figure 1.1.

The system consists of the following main stages:

1. Image acquisition using an industrial camera,
2. Image preprocessing including contrast enhancement and noise reduction,
3. Feature extraction using directional filters,
4. Seam detection and localization,
5. Curve fitting for estimating the seam path,
6. Extraction of key points for control purposes,
7. Transmission of results to the control unit via CAN communication.

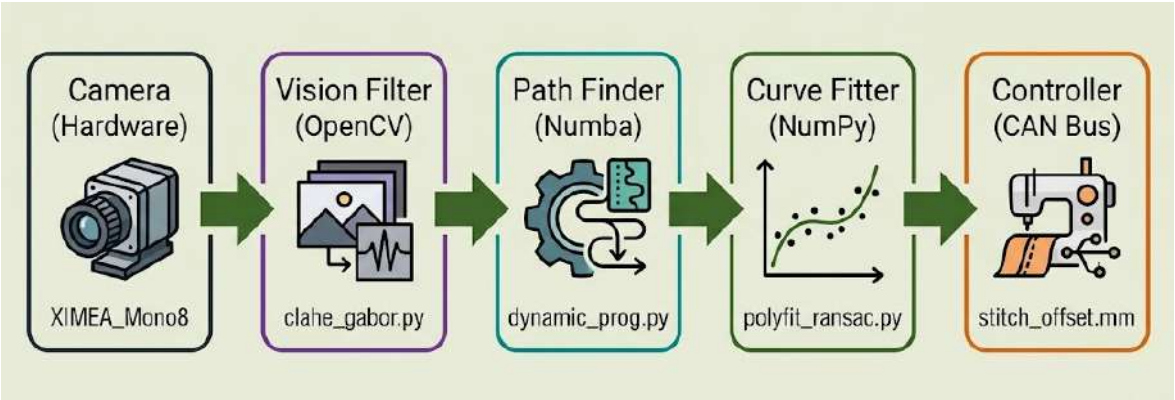


Figure 1.1.: Seam path detection pipeline

1.7. Challenges

The development of a robust seam detection system involves several technical challenges:

- Low contrast between seam and fabric background,
- Variability in fabric textures and patterns,
- Sensitivity to illumination changes,
- Motion blur due to high-speed operation,
- Real-time processing constraints,
- Mechanical vibrations affecting image stability.

Addressing these challenges requires a combination of robust image processing techniques and efficient implementation strategies.

1.8. Outline

This thesis is structured as follows:

Chapter 2 introduces the hardware components and the theoretical concepts underlying the detection system.

Chapter 3 presents the implementation of the image processing pipeline, including seam detection and curve fitting algorithms.

Chapter 4 discusses system calibration, experimental validation, and performance evaluation under real-world conditions.

Finally, Chapter 5 summarizes the findings of this work and outlines possible directions for future research and development.

2. System Concept and Hardware

2.1. Hardware Components

Figure 2.1.: Hardware setup and physical scale.

The detection and control system is built upon several critical hardware components that work in tandem to achieve real-time performance:

1. **Industrial Sewing Machine:** A VETRON industrial sewing machine is utilized to perform the stitching operation.

2. **XIMEA Industrial Camera:** The system uses a mono-colour XIMEA camera. It captures high-resolution fabric images operating specifically in MONO8 mode to optimize the data payload for real-time processing. The physical pixel size conversion is calibrated to 0.0022 mm/pixel.

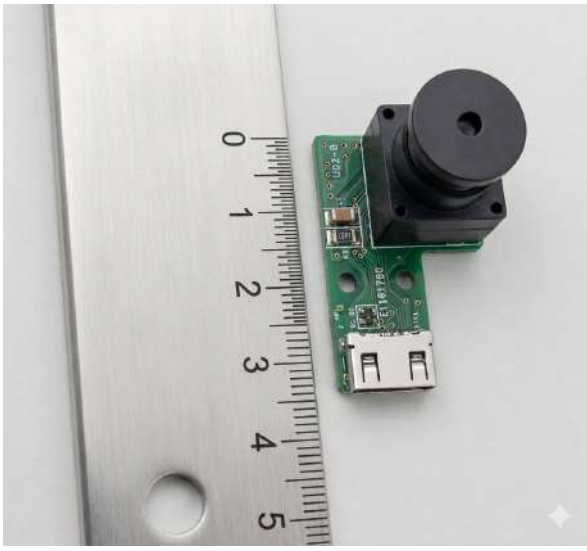


Figure 2.2.: Close-up view of the bare XIMEA camera module.

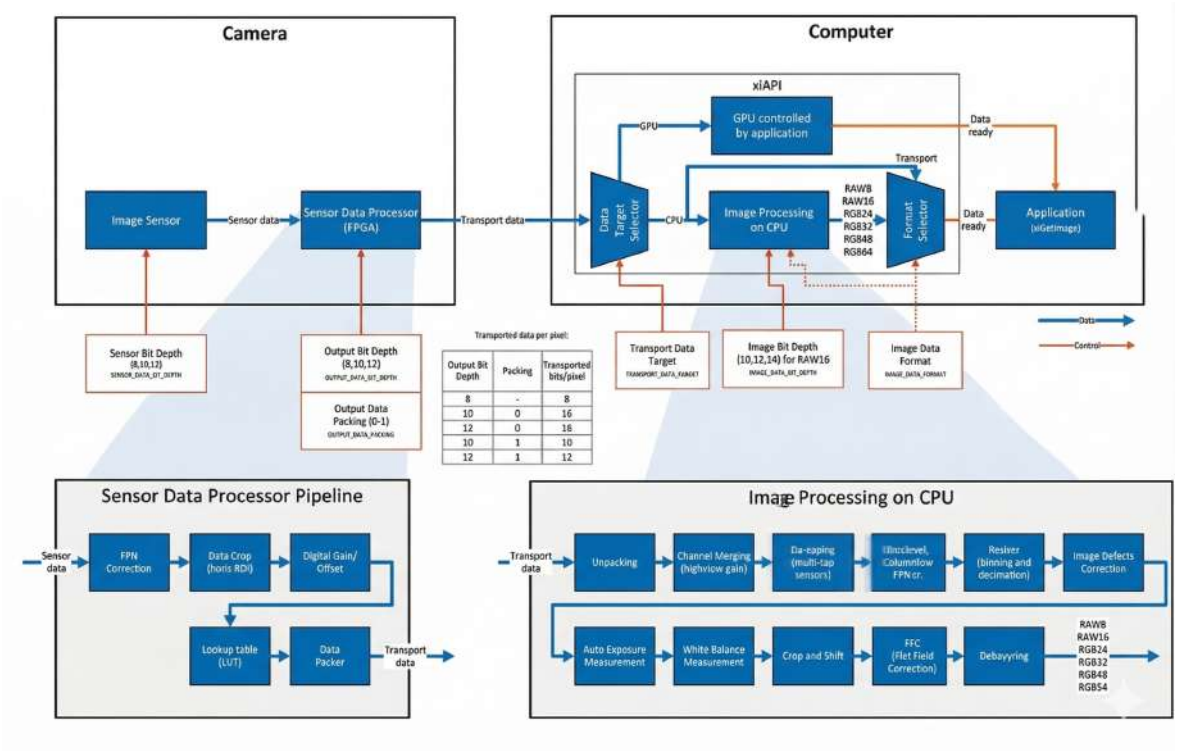


Figure 2.3.: Internal image data flow and processing pipeline of the XIMEA camera via xiAPI [XIM17].

3. **Controlled Lighting Source:** Provides uniform illumination across the fabric to minimize environmental noise and shadows.

4. **Motion Controllers:** These controllers manage the physical fabric movement according to the detected path, though the mechanical actuation itself is outside

the current project scope.

5. **Processor/PC:** A dedicated PC executes the real-time image processing algorithms. The environment relies heavily on Python, utilizing libraries such as OpenCV and NumPy for image manipulation, alongside Numba to compile functions for optimized execution speed.

Figure 2.4.: Side view of the fabric positioned under the sewing machine mechanism.

2.2. Control Concepts and Dynamic Programming

The core of the proposed algorithm is formulated within the framework of discrete optimal control, where seam detection is cast as a sequential decision problem [KCTY23]. The objective is to determine an optimal vertical trajectory (seam) that minimizes a

cumulative cost functional defined over the image domain [KCTY23]. This formulation naturally leads to the use of dynamic programming (DP), which guarantees global optimality under local transition constraints [KCTY23]. Let the grayscale image be interpreted as a cost field, where each pixel (r, c) is associated with an energy value $E(r, c)$ proportional to its intensity [KWB22]. In this implementation, darker pixels correspond to lower costs, enabling the detection of vertical dark structures [KWB22]. The optimal seam is defined as the path minimizing the accumulated cost from the first to the last row [KCTY23]. The cumulative cost matrix $C \in \mathbb{R}^{H \times W}$ is computed recursively as:

$$C(r, c) = E(r, c) + \min(C(r-1, c-1) + P, C(r-1, c), C(r-1, c+1) + P) \quad (2.1)$$

where:

- $E(r, c)$ denotes the local pixel energy derived from the processed image (after CLAHE, Gaussian smoothing, and Gabor filtering) [Sha23, DC20, Kam14].
- P is a diagonal transition penalty, set to 0.05, which regularizes the solution by discouraging abrupt lateral transitions and enforcing trajectory smoothness [KCTY23].

From a control perspective, this formulation can be interpreted as a finite-horizon optimal control problem, where the state corresponds to the column index c at each row r, and the control input is the discrete transition $\Delta c \in \{-1, 0, +1\}$ [KCTY23].

2.2.1. Implementation-Level Optimization

While the theoretical formulation of dynamic programming (DP) guarantees a globally optimal seam path, its practical execution poses a significant computational challenge. The algorithm requires evaluating the Bellman transition equation for every pixel in the image grid. For an image of height H and width W, and an admissible transition space of size $|\mathcal{A}| = 3$, the time complexity of the forward DP stage is strictly $\mathcal{O}(H \cdot W \cdot |\mathcal{A}|)$.

In high-level interpreted languages such as Python, evaluating this recursive function across millions of pixels requires extensive nested loops. Due to dynamic typing and the Global Interpreter Lock (GIL), native Python iteration introduces massive overhead, causing execution times that are completely incompatible with the millisecond-level latencies required for automated sewing [CCP05].

Just-In-Time (JIT) Compilation

To achieve real-time performance without abandoning the rapid development environment of Python, the core dynamic programming recursion is optimized using Numba Just-In-Time (JIT) compilation. Unlike traditional ahead-of-time compilers (like C++ or Rust), a JIT compiler analyzes and translates the code into raw machine instructions at runtime.

When the seam extraction function is called for the first time, the Numba JIT compiler intercepts the Python bytecode. It leverages the LLVM compiler infrastructure to infer the strict static data types of the image arrays (e.g., 8-bit integers or 32-bit floats) and compiles the nested loops directly into highly optimized machine code. Subsequent calls to the function bypass the Python interpreter entirely, executing at speeds comparable to native C. This optimization drastically reduces execution time, enabling high-frequency processing that synchronizes perfectly with real-time image acquisition and control systems [CCP05, KCTY23].

Matrix Memory Allocation

At the implementation level, the algorithm strictly manages memory to avoid costly reallocations during the high-speed camera feed. The JIT-compiled function pre-allocates two fixed-size matrices in memory:

- **Cumulative Cost Matrix ($C \in \mathbb{R}^{H \times W}$):** Stores the running optimal cost-to-go for every pixel.
- **Predecessor Matrix ($M \in \mathbb{Z}^{H \times W}$):** Stores the discrete optimal transition index (e.g., $-1, 0, +1$) that led to the minimum cost at each step.

During the forward pass, these matrices are overwritten in-place rather than instantiated dynamically. Once the optimal endpoint $c^{\star}_{H-1}$ is identified in the bottom row of C, the algorithm rapidly traverses the predecessor matrix M from bottom to top in $\mathcal{O}(H)$ time to reconstruct the final geometric seam trajectory. Figure 2.5 visually summarizes this optimized computational architecture.

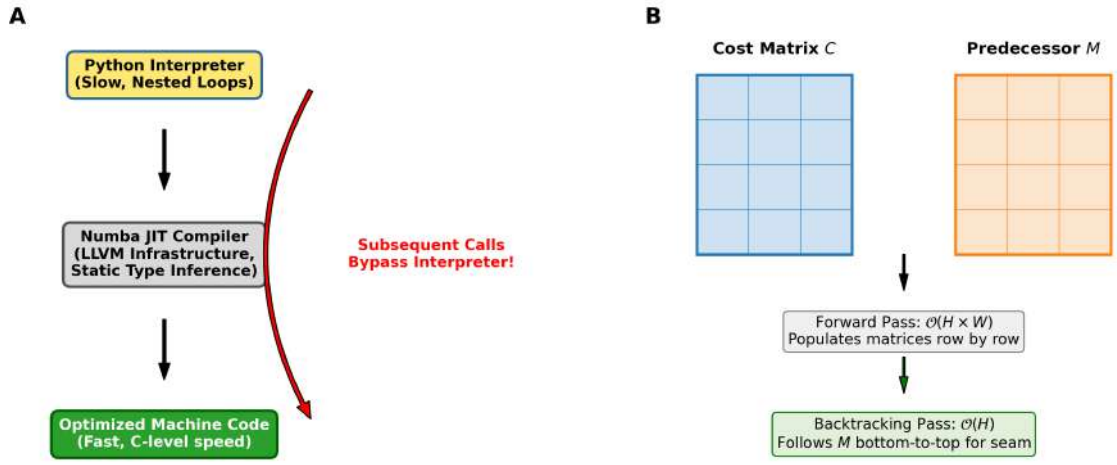


Figure 2.5.: Computational architecture of the optimized DP implementation. (A) The Numba JIT compilation pipeline, bypassing the slow Python interpreter to execute raw machine code. (B) The memory structure during execution, illustrating the parallel updating of the Cumulative Cost Matrix and the Predecessor Matrix, which allows for $\mathcal{O}(H)$ backtracking speed.

2.2.2. Bidirectional Seam Estimation

In standard dynamic programming (DP) algorithms applied to spatial image processing, cumulative costs are propagated uniformly in a single sequential pass. While this guarantees a mathematically optimal path relative to the transition rules, the unidirectional causal nature of the Markov process makes it highly susceptible to "shadowing" or directional bias [KWB22]. If a localized artifact—such as a dark speck of dirt, an uneven fabric fold, or a non-uniform illumination gradient—creates an asymmetric energy well, a single-pass DP algorithm can become trapped in a false local minimum, causing the estimated seam trajectory to permanently diverge from the physical seam [RZR+20].

To improve tracking robustness against these localized artifacts and to explicitly eliminate directional bias, the proposed system employs a bidirectional seam estimation strategy.

Methodological Approach

Instead of relying on a single cumulative cost propagation, the system evaluates the localized energy field from two mirrored perspectives, effectively calculating two independent optimal trajectories.

Let the image width be W, and the baseline Gabor-filtered energy map be denoted

as $E(r, c)$. The bidirectional pipeline executes the following sequence:

1. **Forward Pass (Left-to-Right bias):** The standard DP algorithm evaluates the original energy map E, yielding a primary discrete candidate path:

$$\mathcal{P}_{\text{L2R}} = \{x_{\text{L2R}}(r)\}_{r=0}^{H-1} \tag{2.2}$$

2. **Domain Inversion:** The original energy map is horizontally flipped, creating a mirrored cost field:

$$E_{\text{flip}}(r, c) = E(r, (W-1) - c) \tag{2.3}$$

3. **Reverse Pass (Right-to-Left bias):** The identical DP recursion is executed on E_{flip}, yielding a mirrored path $x_{\text{flip}}(r)$. This path is then mathematically mapped back into the original spatial coordinate system to form the secondary candidate path:

$$x_{\text{R2L}}(r) = (W-1) - x_{\text{flip}}(r) \tag{2.4}$$

Row-Wise Energy Arbitration (Fusion)

Once both x_{L2R} and x_{R2L} are acquired, they often match perfectly along the highly confident segments of the seam. However, in regions containing distracting textures or noise, the two paths will locally diverge. Because the physical seam naturally constitutes the darkest continuous groove in the enhanced image, the true physical location will exhibit a lower mathematical energy.

To synthesize the final robust trajectory, the resulting seam candidates are fused via row-wise energy arbitration. At every row r, the algorithm compares the absolute pixel intensity (cost) of both candidates and dynamically selects the state with the lower energy value [LLL+21]:

$$x_{\text{final}}(r) = \begin{cases} x_{\text{L2R}}(r), & \text{if } E\big(r, x_{\text{L2R}}(r)\big) \leq E\big(r, x_{\text{R2L}}(r)\big) \\ x_{\text{R2L}}(r), & \text{otherwise.} \end{cases} \tag{2.5}$$

This dynamic fusion acts as a powerful element-wise spatial filter. As visualized in Figure 2.6, even if an artifact heavily pulls one path off-course, the opposing path typically remains locked onto the true seam. By arbitrating based strictly on localized minimum energy, the system continuously self-corrects, ensuring that local visual disturbances do not compromise the global trajectory required for accurate machine control.

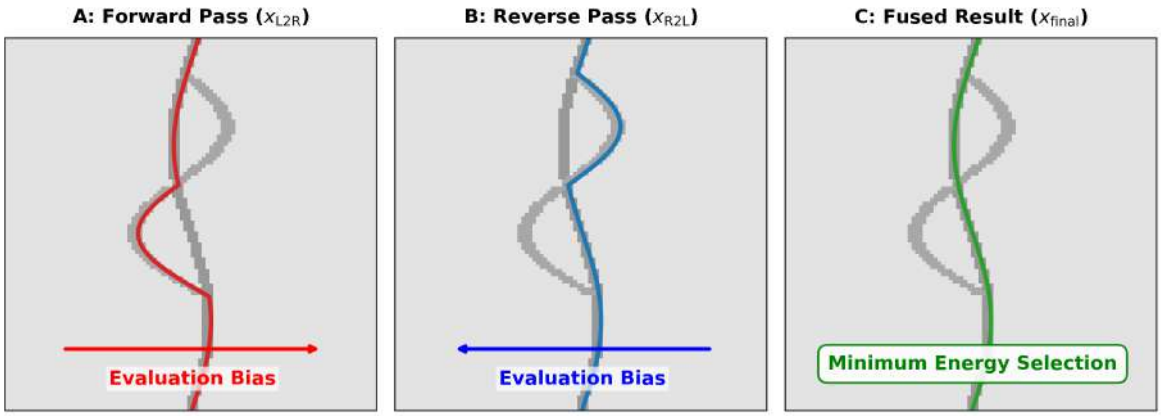


Figure 2.6.: Conceptual visualization of the bidirectional seam estimation strategy. (A) The Forward DP pass is distracted by a dark artifact on the left, causing a path deviation. (B) The Reverse DP pass evaluated on the flipped domain avoids the left artifact but is distracted by a secondary right artifact. (C) Row-wise energy arbitration successfully fuses the two paths by dynamically selecting the minimum local cost, bypassing both artifacts to perfectly track the true physical seam.

2.2.3. Polynomial Trajectory Approximation

Polynomial trajectory approximation is a fundamental mathematical regression technique used to estimate an underlying continuous function from a discrete set of noisy observations. In the context of automated visual inspection and automated guidance, extracting a feature (such as an edge or a seam) typically yields an array of discrete pixel coordinates. Because these coordinates are bound by the finite resolution of the image sensor and are highly susceptible to high-frequency noise, using them directly for machine control often leads to erratic, unstable behavior.

Following seam extraction, the discrete path obtained by the dynamic programming algorithm is approximated using a polynomial model $x = f(y)$ obtained via ordinary least-squares (OLS) fitting [BGH03]. This stage is critical for bridging the gap between discrete computer vision outputs and continuous mechatronic control, providing trajectory smoothing, sub-pixel interpolation, and an analytical representation suitable for downstream processing [BGH03, ZQZ11].

Mathematical Formulation

Let the discrete seam path extracted from the image be defined as a set of N two-dimensional points:

$$\mathcal{P} = \{(y_i, x_i)\}_{i=1}^{N} \tag{2.6}$$

where $y_i \in \{0, 1, \ldots, H-1\}$ represents the independent row index (the longitudinal progression of the fabric), and x_i represents the dependent column index (the lateral deviation of the seam). The goal of polynomial approximation is to find a continuous function $\hat{x}(y)$ of degree k that best models the data:

$$\hat{x}(y) = a_0 + a_1 y + a_2 y^2 + \cdots + a_k y^k = \sum_{j=0}^{k} a_j y^j \tag{2.7}$$

To determine the optimal coefficient vector $\mathbf{a} = [a_0, a_1, \ldots, a_k]^T$, the method of least squares minimizes the sum of squared residuals (SSR) between the observed discrete points and the continuous model:

$$S(\mathbf{a}) = \sum_{i=1}^{N} \left(x_i - \hat{x}(y_i)\right)^2 \tag{2.8}$$

By setting the partial derivatives of $S(\mathbf{a})$ with respect to each coefficient a_j to zero, the problem can be expressed in matrix form as $\mathbf{Ya} = \mathbf{X}$, where:

$$\mathbf{Y} = \begin{bmatrix} 1 & y_1 & y_1^2 & \ldots & y_1^k \\ 1 & y_2 & y_2^2 & \ldots & y_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y_N & y_N^2 & \ldots & y_N^k \end{bmatrix} \quad \text{and} \quad \mathbf{X} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \tag{2.9}$$

The unique analytical solution that minimizes the error is calculated using the Moore-Penrose pseudoinverse:

$$\mathbf{a} = (\mathbf{Y}^T\mathbf{Y})^{-1}\mathbf{Y}^T\mathbf{X} \tag{2.10}$$

Application to Real-Time Seam Tracking

Applying this generalized mathematical model to the specific problem of real-time fabric seam tracking provides three major operational advantages:

1. High-Frequency Noise Suppression (Smoothing): Despite extensive pre-processing (CLAHE, Gaussian, and Gabor filtering), the dynamic programming output often contains minor lateral jitter. This "staircase effect" occurs because the path is constrained to a discrete grid of integer pixels. A polynomial curve naturally acts as a global spatial filter. By strictly limiting the polynomial to a low degree (typically $k \in \{1, 2, 3\}$), the model captures the true macro-geometry of the fabric's curve while mathematically ignoring micro-level pixel fluctuations and localized false detections.

2. Sub-Pixel Interpolation: Standard image processing algorithms limit positional accuracy to whole pixels (e.g., $x = 104$ or $x = 105$). By fitting an analytical curve, the system can evaluate the seam position at any arbitrary row y to yield a continuous floating-point coordinate (e.g., $\hat{x}(y) = 104.37$). This sub-pixel accuracy is vital for smooth closed-loop control of the sewing machine's actuators.

3. Analytical Trajectory Representation: Converting an array of hundreds of points into a single, compact equation with a few coefficients heavily reduces the data footprint. Furthermore, having an analytical function $x = f(y)$ allows the system to instantly compute the derivative $\frac{dx}{dy}$ at any point. This provides the instantaneous tangent (angle) of the seam, which is required if the automated sewing system needs to rotate the needle head to remain perfectly parallel to curved seams.

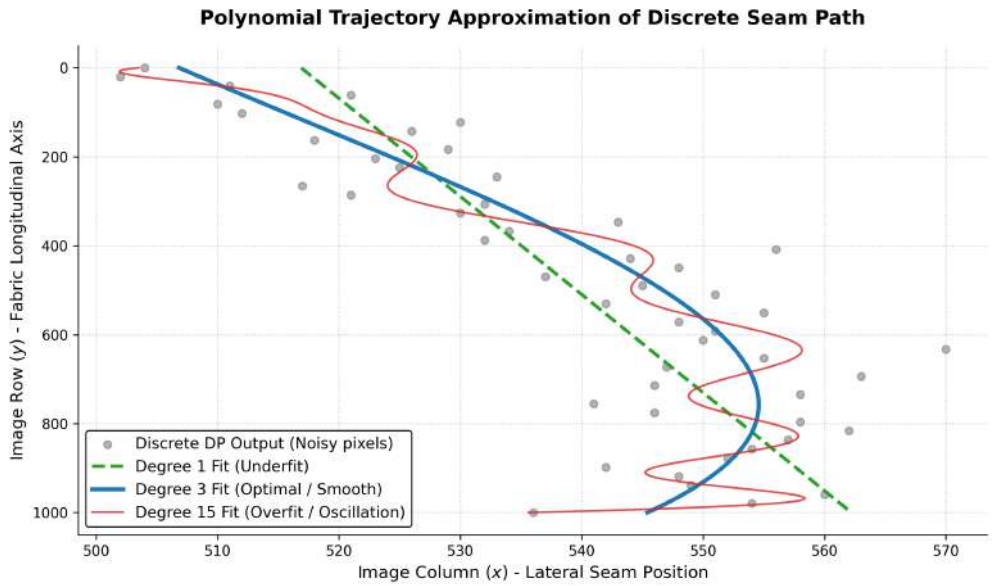


Figure 2.7.: Conceptual illustration of polynomial trajectory approximation on a noisy discrete path. Degree 1 provides a robust linear trend but fails to capture macro-curvature (underfitting). Degree 3 accurately captures the seam geometry while smoothing local pixel noise. Degree 15 demonstrates severe overfitting (Runge's phenomenon), erroneously following high-frequency noise and drastically destabilizing the control trajectory.

Model Selection and Avoiding Overfitting

Choosing the correct polynomial degree k is essential. While a higher-degree polynomial will technically yield a lower fitting error against the training points, it is highly prone to overfitting. As illustrated in Figure 2.7, an excessively high-degree polynomial (e.g., $k = 15$) begins to fit the high-frequency pixel noise rather than the physical seam

path, resulting in wild spatial oscillations—a mathematical artifact known as Runge's phenomenon.

To ensure robust machine behavior, the implemented system evaluates low-order polynomials ($k = 1, 2, 3$) and dynamically selects the model that minimizes the Mean Squared Error (MSE) without introducing higher-order instability.

2.3. Development Methodology

The project follows the V-Model methodology for systems engineering, ensuring a structured approach from high-level design to low-level implementation and testing [KCTY23].

- **Design and Implementation:** The descending branch begins with System Analysis and System Design (utilizing Model-in-the-Loop), moving into Software Design using Rapid Control Prototyping.
- **Automatic Code Generation:** The base of the "V" involves On-Target Rapid Control Prototyping and the core implementation phase [KCTY23].
- **Validation and Verification:** The ascending branch progresses from Software Integration (Software-in-the-Loop) to Hardware/Software Integration (Processor-in-the-Loop), concluding with full System Integration via Hardware-in-the-Loop testing [KCTY23].

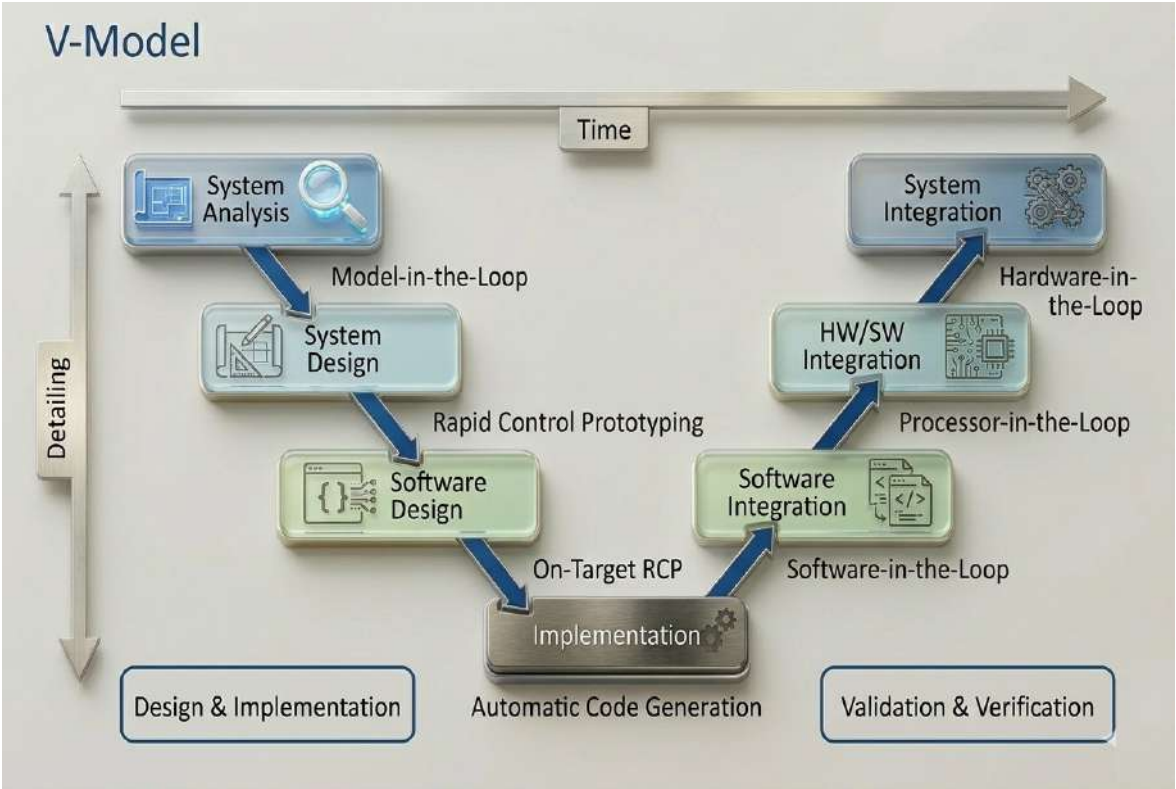


Figure 2.8.: The V-Model methodology applied to the project development.

3. Implementation

3.1. Image Processing Method

The proposed system is implemented as a real-time image processing pipeline designed to extract seam paths from grayscale fabric imagery [CCP05, RZR+20, MKP09]. Images are acquired using a XIMEA industrial camera operating in MONO8 mode, which provides 8-bit grayscale intensity values in the range $[0, 255]$. This representation is particularly suitable for cost-based seam detection, since darker pixels correspond to lower-cost regions.

The objective of the pipeline is to transform the raw image into a representation where the seam appears as a continuous and distinguishable structure, while suppressing noise, illumination variations, and irrelevant textures [KYMP05]. The processing pipeline consists of the following stages:

1. Image acquisition in MONO8 grayscale format,
2. local contrast enhancement using CLAHE,
3. Gaussian smoothing for noise reduction,
4. Gabor filtering for texture-oriented enhancement.

Each stage incrementally improves seam visibility and continuity.

3.1.1. Image Acquisition

Let the captured image be denoted as

$$I_0(r, c), \quad r = 0, \ldots, H-1, \; c = 0, \ldots, W-1 \tag{3.1}$$

where H and W represent the image height and width, respectively. In the implemented setup, the acquired frame is rotated by 180° in order to maintain a consistent coordinate system for downstream processing and measurement.

The XIMEA camera is initialized through the manufacturer API and configured to operate in grayscale mode [XIM17]. This choice simplifies the subsequent processing stages, since the seam extraction algorithm directly interprets pixel intensity as a cost value. Furthermore, grayscale acquisition reduces computational complexity compared to RGB processing while preserving the relevant structural information of the fabric seam.

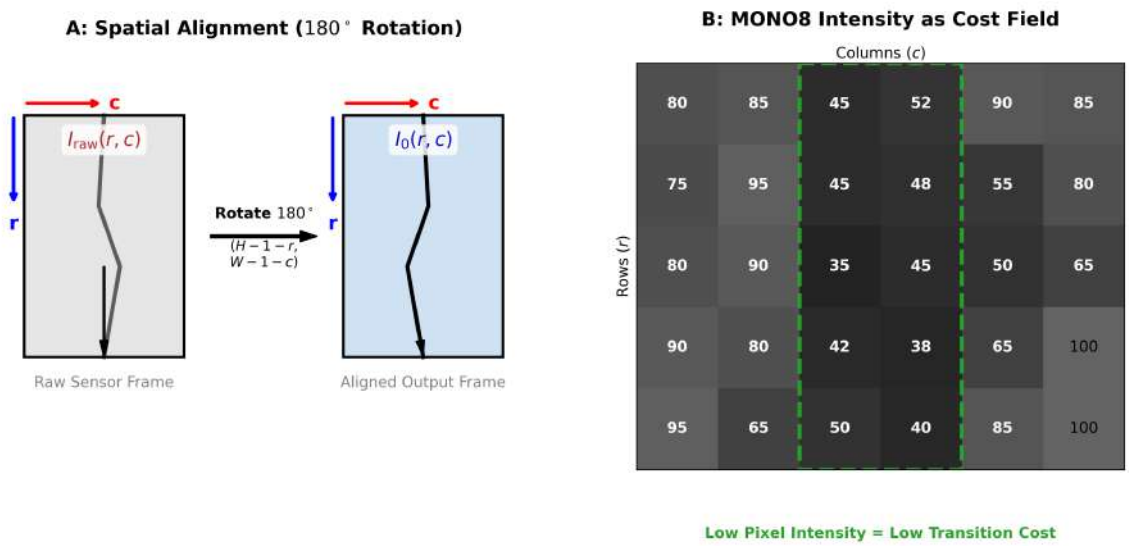


Figure 3.1.: Conceptual overview of the image acquisition stage. (A) The raw sensor image is rotated 180° via an affine transformation to physically align the image matrix coordinate system (r, c) with the fabric feed direction. (B) The MONO8 grayscale acquisition maps physical brightness directly to an 8-bit integer matrix, naturally forming a cost field where the dark physical seam corresponds to localized mathematical minimums.

3.1.2. CLAHE-Based Contrast Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) is employed as the first stage of the image enhancement pipeline [Sha23, VJDBSS22]. CLAHE is a locally adaptive contrast enhancement technique that operates on small contextual regions (tiles), rather than applying a single global transformation over the entire image. This makes it particularly suitable for fabric imagery, where seam structures are often only locally distinguishable due to non-uniform illumination, shadows, and material inhomogeneities.

Let the input grayscale image be defined as

$$I_0 : \Omega \subset \mathbb{R}^2 \to [0, L-1], \tag{3.2}$$

where L denotes the number of intensity levels. The CLAHE-enhanced image is given

by

$$I_{\text{CLAHE}}(r, c) = H_{\text{CLAHE}}(I_0(r, c)), \tag{3.3}$$

where $H_{\text{CLAHE}}(\cdot)$ represents the adaptive histogram equalization operator with contrast limiting.

Local Histogram Equalization

The image domain is divided into non-overlapping tiles $T_{i,j}$ of size $M \times N$. For each tile, a local histogram is computed as

$$h_{i,j}(k) = \#\{(r, c) \in T_{i,j} \mid I_0(r, c) = k\}. \tag{3.4}$$

The corresponding cumulative distribution function (CDF) is defined as

$$\text{CDF}_{i,j}(k) = \sum_{m=0}^{k} h_{i,j}(m). \tag{3.5}$$

Each pixel intensity is remapped using the normalized CDF:

$$I_{i,j}(r, c) = \frac{L-1}{|T_{i,j}|} \cdot \text{CDF}_{i,j}(I_0(r, c)). \tag{3.6}$$

This local transformation enhances contrast within each tile, allowing subtle intensity variations—such as seam structures—to become more pronounced.

Contrast Limiting

A key limitation of Adaptive Histogram Equalization (AHE) is the amplification of noise in homogeneous regions. CLAHE addresses this by introducing a clip limit C:

$$h'_{i,j}(k) = \min\left(h_{i,j}(k), C\right). \tag{3.7}$$

The total clipped excess is computed as

$$E_{i,j} = \sum_{k=0}^{L-1} \left(h_{i,j}(k) - h'_{i,j}(k)\right), \tag{3.8}$$

and redistributed uniformly across all bins:

$$h''_{i,j}(k) = h'_{i,j}(k) + \frac{E_{i,j}}{L}. \tag{3.9}$$

This mechanism ensures controlled contrast enhancement while preventing the over-amplification of noise and avoiding the introduction of artificial edges.

Interpolation Between Tiles

To eliminate artificial boundaries between adjacent tiles, bilinear interpolation is applied [VJDBSS22]. The final pixel intensity is obtained as a weighted combination of neighboring tile mappings:

$$I_{\text{CLAHE}}(r, c) = \sum_{p,q} w_{p,q}(r, c)\, I_{p,q}(r, c), \tag{3.10}$$

subject to

$$\sum_{p,q} w_{p,q}(r, c) = 1. \tag{3.11}$$

This ensures smooth transitions across tile boundaries and preserves spatial consistency.

Comparison with Global Equalization

Figure 3.2 illustrates the difference between global histogram equalization and local adaptive equalization. While global methods enhance overall contrast, they often fail to highlight local structures and may over-compress intensity distributions. In contrast, CLAHE enhances local details while maintaining a balanced intensity distribution.

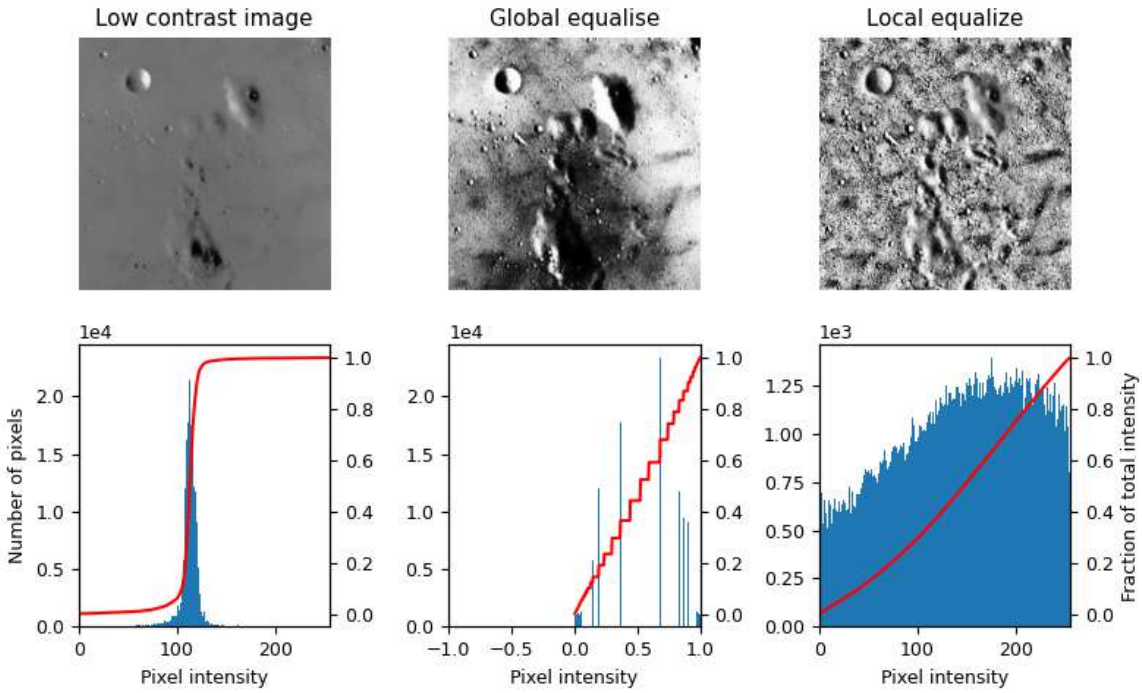


Figure 3.2.: Comparison between low-contrast image, global histogram equalization, and local adaptive equalization (CLAHE). The corresponding histograms and cumulative distributions are shown below each image. CLAHE enhances local contrast while avoiding over-amplification of noise.

Parameter Selection and Implementation

In the proposed system, CLAHE is implemented using OpenCV with the following parameters:

- Clip limit: $C = 2.0$
- Tile grid size: 8×8

These values provide a suitable trade-off between local contrast enhancement and noise suppression. The transformation is applied as a preprocessing step before Gaussian smoothing and Gabor filtering.

Impact on Seam Detection

The application of CLAHE significantly improves the visibility of seam-related intensity variations, particularly under challenging illumination conditions. By locally redistributing intensity values, weak seam structures become more distinguishable from the surrounding fabric.

Furthermore, the contrast-limiting mechanism prevents the creation of artificial edges in homogeneous regions, which is critical for avoiding false detections. This leads to a more stable and reliable energy representation for subsequent processing stages, including Gabor-based texture enhancement and dynamic programming seam extraction.

Overall, CLAHE enhances the robustness of the entire pipeline by increasing the signal-to-noise ratio of seam features while preserving structural consistency.

Quantitative Justification Using Contrast Metrics

To objectively evaluate the effectiveness of CLAHE, several quantitative contrast metrics can be considered. These metrics provide a numerical assessment of how the transformation improves the visibility of seam structures.

Global Contrast (Standard Deviation) A common measure of image contrast is the standard deviation of intensity values:

$$\sigma_I = \sqrt{\frac{1}{|\Omega|} \sum_{(r,c)\in\Omega} \left(I(r,c) - \mu_I\right)^2} \tag{3.12}$$

where μ_I is the mean intensity:

$$\mu_I = \frac{1}{|\Omega|} \sum_{(r,c)\in\Omega} I(r,c). \tag{3.13}$$

An increase in σ_I after applying CLAHE indicates improved global contrast. However, this metric alone is insufficient for evaluating seam visibility, since CLAHE primarily enhances *local* contrast.

Local Contrast Local contrast is more relevant for seam detection and can be defined over a neighborhood $\mathcal{N}(r,c)$ as:

$$C_{\text{local}}(r,c) = \frac{\max_{\mathcal{N}} I - \min_{\mathcal{N}} I}{\max_{\mathcal{N}} I + \min_{\mathcal{N}} I + \epsilon} \tag{3.14}$$

where ϵ is a small constant to avoid division by zero.

CLAHE significantly increases C_{local} in regions containing seam structures, making them more distinguishable from the background fabric.

Michelson Contrast (Seam-to-Background) For periodic or structured patterns such as seams, the Michelson contrast is particularly meaningful:

$$C_M = \frac{I_{\max} - I_{\min}}{I_{\max} + I_{\min}} \tag{3.15}$$

In the context of seam detection:

- $I_{\max}$ corresponds to background fabric intensity
- $I_{\min}$ corresponds to seam intensity

An increase in C_M after CLAHE indicates improved separability between seam and background.

Entropy-Based Measure Image entropy quantifies the amount of information contained in the intensity distribution:

$$H = -\sum_{k=0}^{L-1} p(k) \log p(k) \tag{3.16}$$

where $p(k)$ is the normalized histogram.

CLAHE typically increases entropy by redistributing intensity values more uniformly, indicating a richer representation of local details.

Impact on Seam Detection Energy Function In the proposed system, seam detection is formulated as an energy minimization problem, where the cost function depends on pixel intensities:

$$J = \sum_r I(r, x_r) + \lambda \cdot \text{penalty}. \tag{3.17}$$

By enhancing local contrast, CLAHE increases the intensity difference between seam and non-seam regions. This leads to:

- clearer minima in the energy landscape
- reduced ambiguity during path optimization
- improved stability of seam localization

Summary Overall, CLAHE improves both global and local contrast metrics, increases entropy, and enhances seam-to-background separability. These effects directly contribute to a more reliable and robust seam detection process.

3.1.3. Gaussian Smoothing

Following local contrast enhancement, Gaussian smoothing is applied to attenuate high-frequency noise and small-scale intensity fluctuations that may negatively affect subsequent processing stages [Get13, PSS20, DC20, Yu23]. This step is particularly important in fabric imagery, where fine texture patterns, fiber irregularities, and sensor noise introduce spurious gradients that can interfere with reliable seam detection [SPYJ05].

Gaussian Filtering: Continuous Formulation

Let I_{CLAHE} denote the contrast-enhanced image. The smoothed image I_{gauss} is obtained via convolution with a Gaussian kernel:

$$I_{\text{gauss}}(r, c) = (I_{\text{CLAHE}} * G_\sigma)(r, c) \tag{3.18}$$

where $*$ denotes the two-dimensional convolution operator, and the Gaussian kernel is defined as:

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{3.19}$$

The Gaussian function is isotropic and separable, meaning it can be expressed as:

$$G_\sigma(x, y) = G_\sigma(x)\, G_\sigma(y) \tag{3.20}$$

This separability allows efficient implementation with reduced computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(2N)$.

Discrete Convolution Formulation

In practice, convolution is implemented in discrete form over a finite kernel window K:

$$I_{\text{gauss}}(r, c) = \sum_{(i,j)\in K} G_\sigma(i, j)\, I_{\text{CLAHE}}(r - i, c - j) \tag{3.21}$$

The kernel weights satisfy the normalization constraint:

$$\sum_{(i,j)\in K} G_\sigma(i, j) = 1 \tag{3.22}$$

This ensures that the overall image brightness is preserved after filtering.

Frequency Domain Interpretation

The Gaussian filter acts as an ideal low-pass filter in the frequency domain. Its Fourier transform is also Gaussian:

$$\mathcal{F}\{G_\sigma\}(\omega_x, \omega_y) = \exp\left(-\frac{\sigma^2(\omega_x^2 + \omega_y^2)}{2}\right) \tag{3.23}$$

This property implies:

- High-frequency components (noise, fine texture) are attenuated
- Low-frequency components (global structures, seam trajectory) are preserved

This behavior is essential for stabilizing gradient-based and energy-based processing stages.

Scale-Space Interpretation

Gaussian smoothing is fundamentally linked to scale-space theory, where an image is represented at multiple levels of resolution:

$$L(r, c, \sigma) = (I_{\text{CLAHE}} * G_\sigma)(r, c) \tag{3.24}$$

The parameter σ defines the observation scale:

- Small σ: preserves fine details but retains noise
- Large σ: suppresses noise but may blur structural features

Thus, Gaussian filtering provides a principled way to control the trade-off between noise suppression and structural preservation.

Parameter Selection and Implementation

In the proposed system, Gaussian smoothing is implemented using OpenCV as:

```
img_blur = cv2.GaussianBlur(frame_clahe, (9, 9), 0)
```

with the following configuration:

- Kernel size: 9×9
- Standard deviation: σ implicitly derived from kernel size

The chosen kernel size provides sufficient spatial support to suppress local intensity fluctuations while preserving the continuity of seam structures. A smaller kernel would fail to remove texture-induced noise, whereas a larger kernel could overly smooth the seam and degrade localization accuracy.

Role Within the Processing Pipeline

Gaussian smoothing is applied after CLAHE and before Gabor filtering:

$$I_{\text{CLAHE}} \to I_{\text{gauss}} \to I_{\text{gabor}} \tag{3.25}$$

This ordering is critical:

- CLAHE enhances local contrast but may amplify noise
- Gaussian filtering suppresses this amplified noise
- Gabor filtering then extracts oriented seam features on a stabilized signal

Thus, Gaussian smoothing acts as a regularization step between contrast enhancement and feature extraction.

Impact on Seam Detection

The seam detection algorithm employed in this work is based on cumulative cost minimization, where the energy function is derived directly from image intensity:

$$J = \sum_r I(r, x_r) + \lambda \cdot \text{penalty} \tag{3.26}$$

Without smoothing, high-frequency noise introduces:

- spurious local minima in the cost map
- unstable path transitions
- sensitivity to texture variations

Gaussian filtering mitigates these effects by enforcing spatial coherence in the intensity field. As a result:

- the energy landscape becomes smoother
- seam paths become more stable across rows
- false detections are significantly reduced

Importantly, the Gaussian filter preserves the global structure of the seam, ensuring that its trajectory remains detectable even after noise suppression.

Practical Effect in Fabric Imagery

In fabric inspection scenarios, the dominant disturbances arise from:

- repetitive texture patterns
- illumination variations
- sensor noise

Gaussian smoothing selectively attenuates these disturbances while preserving the larger-scale seam geometry. This balance is crucial for achieving reliable real-time seam detection and stable polynomial fitting in subsequent stages.

Frequency Spectrum Analysis Before and After Smoothing

To further analyze the effect of Gaussian smoothing, the image can be examined in the frequency domain using the Fourier Transform. Let the 2D discrete Fourier transform of the image be defined as:

$$F(u,v) = \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} I(r,c)\, e^{-j2\pi\left(\frac{ur}{M}+\frac{vc}{N}\right)} \tag{3.27}$$

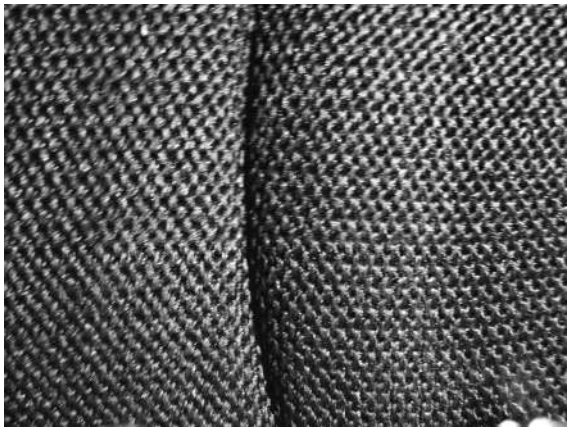
The magnitude spectrum is given by:

$$|F(u,v)| = \sqrt{\Re(F)^2 + \Im(F)^2} \tag{3.28}$$

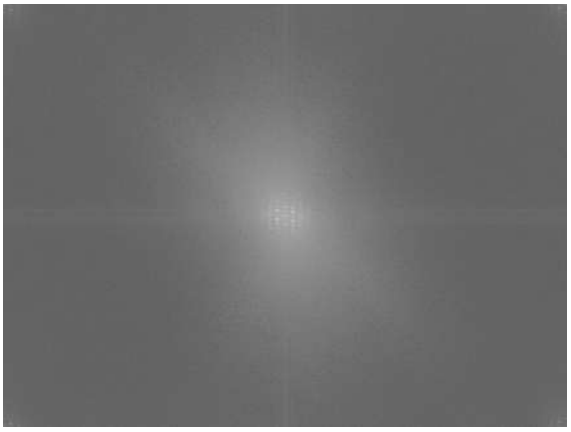
In the context of fabric imagery, high-frequency components correspond to noise, fine fabric micro-textures, and sharp local intensity variations. Conversely, low-frequency components correspond to the global illumination structure and the broader geometric trajectory of the seam itself.

Figure 3.3 illustrates the dual effect of Gaussian smoothing in both the spatial and frequency domains. Before filtering, the raw spatial image exhibits prominent fabric weave patterns. Its corresponding magnitude spectrum reveals strong high-frequency energy distributed outward from the center.

After applying the Gaussian filter, the spatial image becomes visibly smoother, successfully attenuating the abrasive micro-textures while preserving the darker, continuous path of the seam. This low-pass behavior is mathematically confirmed in the resulting frequency spectrum: the high-frequency components in the outer regions are significantly suppressed, while the essential low-frequency structural components in the central region remain intact. This stabilization is crucial for preventing the dynamic programming algorithm from getting trapped in local minima caused by fabric noise.

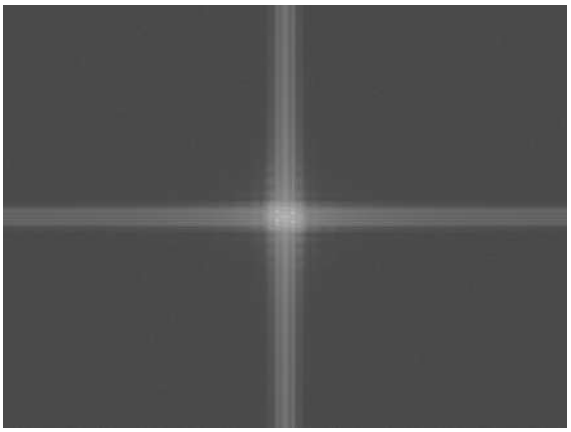


(a) Spatial image before smoothing



(b) Frequency spectrum before smoothing

(c) Spatial image after smoothing



(d) Frequency spectrum after smoothing

Figure 3.3.: Effect of Gaussian smoothing analyzed in both spatial and frequency domains. The Gaussian filter successfully suppresses high-frequency noise and fine fabric textures (visible in the outer regions of the frequency spectra) while preserving the low-frequency global structure of the seam path.

Summary

Gaussian smoothing serves as a critical intermediate step that regularizes the image representation. By suppressing high-frequency noise while preserving structural continuity, it enhances the robustness of downstream processing stages, ultimately enabling accurate and consistent seam localization.

3.1.4. Gabor Filtering for Texture Enhancement

To further emphasize seam-related structures, a Gabor filter is applied after Gaussian smoothing. Gabor filters are particularly well suited for texture analysis because they provide joint localization in both spatial and frequency domains, while also being highly selective to orientation [Kam14, BF07, DH95]. This makes them especially effective for

fabric inspection tasks, where seam patterns typically exhibit a dominant directional structure embedded within repetitive background textures [JYLK14, CZGL20, ED24].

Let I_{gauss} denote the smoothed image. The Gabor-filtered image is obtained by convolution with a two-dimensional Gabor kernel:

$$I_{\text{gabor}}(r, c) = (I_{\text{gauss}} * g)(r, c) \tag{3.29}$$

Gabor Kernel Formulation The real-valued two-dimensional Gabor kernel is defined as:

$$g(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi \frac{x'}{\lambda} + \psi\right) \tag{3.30}$$

with rotated coordinates:

$$x' = x\cos\theta + y\sin\theta, \qquad y' = -x\sin\theta + y\cos\theta \tag{3.31}$$

This formulation represents a sinusoidal plane wave modulated by a Gaussian envelope, resulting in a band-pass filter that is localized in both space and frequency.

Discrete Convolution Implementation In discrete form, the filtering operation is expressed as:

$$I_{\text{gabor}}(r, c) = \sum_{(i,j)\in K} g(i, j)\, I_{\text{gauss}}(r - i, c - j) \tag{3.32}$$

where K denotes the finite kernel support. To ensure numerical stability, the kernel is typically normalized such that:

$$\sum_{(i,j)\in K} |g(i, j)| = 1 \tag{3.33}$$

Frequency and Orientation Selectivity The Gabor filter acts as an oriented band-pass filter centered around a spatial frequency $f_0 = \frac{1}{\lambda}$. Its frequency response is concentrated along a direction defined by θ, allowing selective enhancement of structures aligned with this orientation. As a result, textures matching the filter orientation and wavelength are amplified, while others are attenuated.

Parameter Interpretation The behavior of the Gabor filter is governed by several key parameters:

- θ: orientation of the filter. It determines the direction of sensitivity, allowing the filter to align with the seam trajectory.
- λ: wavelength of the sinusoidal component. It controls the scale of texture patterns to be enhanced.
- σ: standard deviation of the Gaussian envelope. It defines the spatial extent and bandwidth of the filter.
- γ: aspect ratio. It controls the ellipticity of the Gaussian envelope, influencing orientation selectivity.
- ψ: phase offset. It determines the symmetry of the filter response (even or odd).

The interplay of these parameters and the fundamental construction of the Gabor filter are visually summarized in Figure 3.4 [DLPK15]. As illustrated in Figure 3.4(B), the 1D mathematical decomposition clearly demonstrates how the localized Gaussian envelope restricts the infinite sinusoidal carrier, solving the spatial localization problem.

When extended to two dimensions, varying the wavelength (λ) and orientation (θ) creates a highly specific directional filter bank, as shown in the spatial parameter sweep in Figure 3.4(A). Finally, Figure 3.4(C) conceptualizes the resulting behavior in the frequency domain. Unlike the Gaussian filter, which acts as a non-directional low-pass filter at the center of the spectrum, the Gabor filter acts as an off-center, directional band-pass filter. This unique frequency response is what allows the system to aggressively isolate the specific orientation and frequency of the fabric seam while suppressing irrelevant background textures.

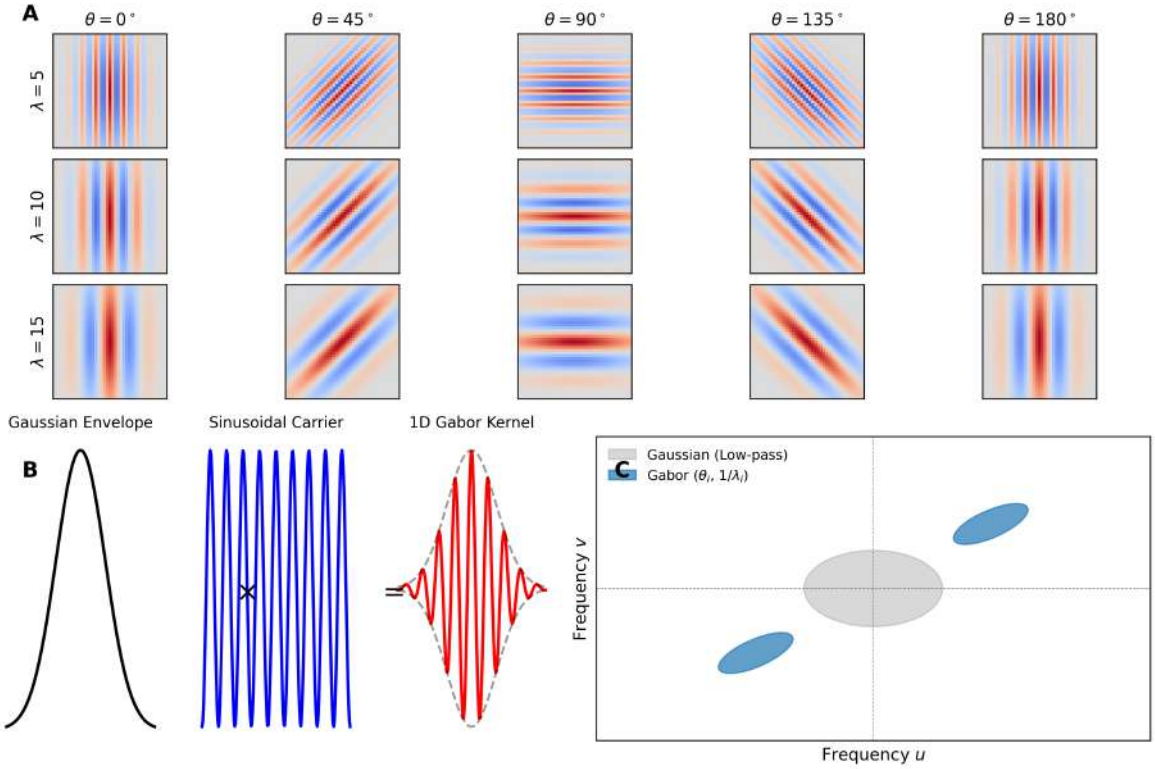


Figure 3.4.: Conceptual overview of the 2D Gabor filter. (A) Spatial domain parameter sweep demonstrating the effect of varying orientation (θ) and wavelength (λ). (B) 1D mathematical decomposition illustrating the multiplication of a sinusoidal carrier by a Gaussian envelope for spatial localization. (C) Conceptual 2D frequency response, contrasting the central low-pass behavior of a Gaussian filter with the directional, localized band-pass behavior of a Gabor filter.

A useful relationship between parameters is given by the approximate bandwidth B of the filter:

$$B \approx \frac{\lambda}{\sigma} \tag{3.34}$$

which indicates that increasing σ results in a narrower frequency bandwidth, improving frequency selectivity at the expense of spatial localization.

Kernel Size and Spatial Support The kernel size determines the effective spatial support of the filter and should be chosen large enough to capture the Gaussian envelope. In practice, the kernel size K_s is typically selected as:

$$K_s \approx 6\sigma + 1 \tag{3.35}$$

Figure 3.5 visually explains the mathematical justification for this formulation. Sta-

tistically, approximately 99.7% of the energy of a Gaussian distribution lies within three standard deviations ($\pm 3\sigma$) from the center.

Truncating the filter matrix at these boundaries (resulting in a total width of 6σ) ensures that the Gabor filter's envelope has decayed to near-zero. Using a smaller kernel size would abruptly cut off the sinusoidal wave, introducing severe frequency-domain artifacts (ringing effect). Conversely, using a kernel larger than 6σ would quadratically increase the computational cost $\mathcal{O}(K_s^2)$ without providing any meaningful additional structural information, as the filter values in those outer regions are effectively zero. The $+1$ term simply ensures that the discrete matrix dimension is an odd number, providing the filter with an exact geometric center pixel.

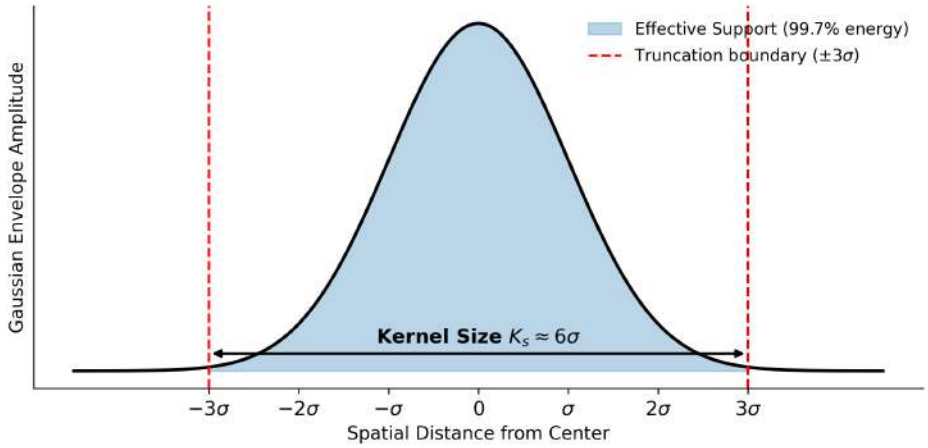


Figure 3.5.: Visual justification for the Gabor kernel spatial support. Setting the kernel width to 6σ safely captures the effective area of the Gaussian envelope without introducing truncation artifacts or unnecessary computational overhead.

to ensure that the Gaussian envelope decays sufficiently at the boundaries. A larger kernel improves frequency resolution but increases computational cost.

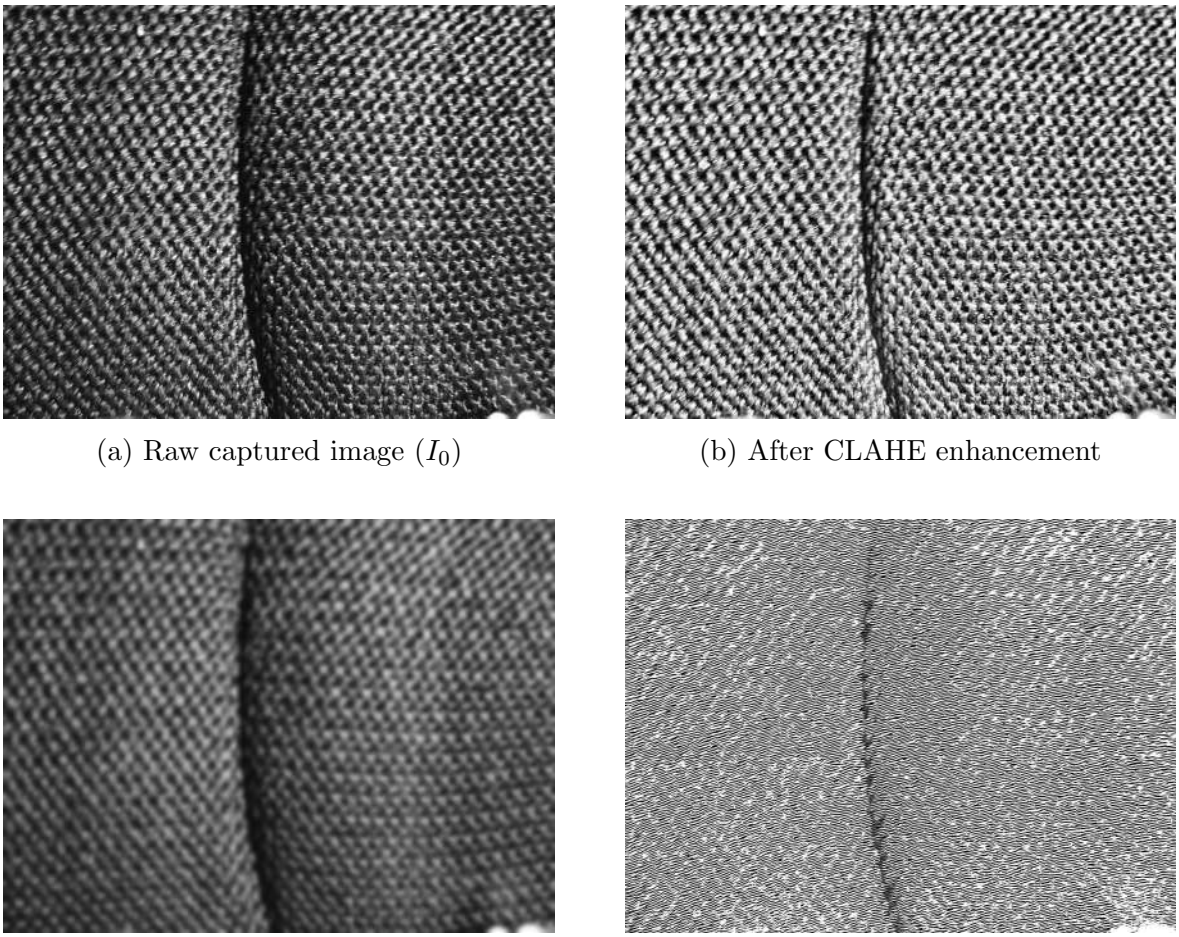
Parameter Selection and Impact In the present implementation, the Gabor kernel is parameterized with kernel size 31, $\sigma = 7.0$, $\theta = \frac{\pi}{2} - 0.25$, $\lambda = 10.0$, $\gamma = 0.5$, and $\psi = 0$. These values were selected to align the filter with the dominant seam orientation while capturing the characteristic spatial frequency of the seam texture.

The relatively low aspect ratio ($\gamma = 0.5$) increases orientation selectivity, allowing the filter to suppress background textures that are not aligned with the seam. The chosen wavelength λ ensures that the periodic seam structure is emphasized, while the Gaussian envelope controlled by σ provides sufficient spatial localization.

Impact on Seam Detection This stage enhances seam-aligned textures while attenuating irrelevant background patterns, resulting in a more discriminative representation for subsequent processing. In particular, the Gabor filter strengthens coherent directional features, which contributes to a smoother and more stable cost map for seam extraction. This reduces ambiguity in path estimation and improves robustness against complex fabric textures [GPK02].

3.1.5. Summary of Preprocessing Effects

To visually summarize the sequential impact of the implemented image processing pipeline, Figure 3.6 illustrates the step-by-step transformation of a fabric sample.



(a) Raw captured image (I_0)

(b) After CLAHE enhancement

(c) After Gaussian smoothing

(d) After Gabor filtering

Figure 3.6.: Sequential visualization of the image processing pipeline demonstrating the isolation of the seam path.

As observed in Figure 3.6(a), the raw captured image often suffers from low global contrast and illumination inconsistencies, making the physical seam difficult to distinguish purely by pixel intensity. The application of CLAHE, shown in Figure 3.6(b),

drastically improves the local contrast. It successfully brings out the structural depth of the seam; however, it also inevitably amplifies the high-frequency noise of the surrounding fabric weave.

To counteract this, the Gaussian blur shown in Figure 3.6(c) acts as a low-pass filter. It smooths out the abrasive high-frequency fluctuations introduced by the contrast enhancement, rendering the seam region more spatially coherent while suppressing the fabric's micro-textures. Finally, Figure 3.6(d) demonstrates the output of the Gabor filter. By tuning the filter to the specific frequency and orientation of the stitch, the background fabric is aggressively suppressed. The resulting image presents the seam as a highly contrasted, continuous dark strip. This final representation perfectly conditions the image array for the dynamic programming algorithm, as the true physical seam now corresponds directly to the lowest-cost mathematical path.

3.2. Seam Path Detection and Fitting

After preprocessing, the seam path is extracted using a cost-based optimization method grounded in dynamic programming. The seam is assumed to correspond to a dark and spatially continuous structure; therefore, pixel intensity is used directly as a local cost measure.

3.2.1. Dynamic Programming

Dynamic programming (DP) is a classical optimization technique for solving multistage decision problems by decomposing them into a sequence of simpler subproblems. In optimal control, the central idea is that the globally optimal solution can be obtained by recursively combining locally optimal transitions according to Bellman's principle of optimality. For a discrete system, this principle states that an optimal trajectory from an initial state to a final state is composed of optimal sub-trajectories. In the present work, this concept is adapted to seam extraction by interpreting the seam centerline as an optimal path through the image from the top row to the bottom row.

Let the image be represented on a discrete grid with row index $r \in \{0, \ldots, H-1\}$ and column index $c \in \{0, \ldots, W-1\}$, where H and W denote the image height and width, respectively. After contrast enhancement and Gaussian smoothing, a Gabor filter is applied to emphasize seam-related texture patterns. Based on the filtered image, a local energy map is defined as

$$E(r, c) = I_{\text{gabor}}(r, c), \tag{3.36}$$

where $I_{\text{gabor}}(r, c)$ denotes the intensity value at pixel (r, c) in the Gabor-filtered image. Since the target seam appears as a dark structure, pixels with lower intensity correspond to lower energy and are therefore preferred by the optimization process.

The seam extraction problem is then formulated as a discrete shortest-path problem on a layered graph, where each image row represents one stage of the optimization. At each row, the seam is allowed to move to one of three neighboring positions in the next row: vertically downward, diagonally downward-left, or diagonally downward-right. Thus, the admissible state transitions satisfy

$$c_r \in \{c_{r-1} - 1,\ c_{r-1},\ c_{r-1} + 1\}. \tag{3.37}$$

This constrained motion model enforces continuity of the extracted seam and prevents unrealistic jumps between distant columns.

To compute the optimal path, a cumulative cost matrix $C(r, c)$ is defined recursively. The first row is initialized as

$$C(0, c) = E(0, c), \tag{3.38}$$

and for all subsequent rows the cumulative cost is computed as

$$C(r, c) = E(r, c) + \min \begin{Bmatrix} C(r-1, c-1) + P, \\ C(r-1, c), \\ C(r-1, c+1) + P \end{Bmatrix}, \tag{3.39}$$

where P is a diagonal penalty term introduced to discourage excessive lateral motion. In the current implementation, this penalty is set to

$$P = 0.05. \tag{3.40}$$

The penalty acts as a regularization term: purely vertical continuation is favored, while diagonal steps are still permitted when supported by significantly lower local image energy. This improves robustness by reducing undesirable zigzag behavior and producing smoother seam trajectories.

From an optimal control perspective, the row index can be interpreted as a discrete stage variable, while the column position represents the system state. The seam search may therefore be understood as the minimization of the cumulative objective

$$J = \sum_{r=0}^{H-1} E(r, c_r) + \sum_{r=1}^{H-1} \rho(c_r - c_{r-1}), \tag{3.41}$$

where the transition penalty function $\rho(\cdot)$ is defined as

$$\rho(c_r - c_{r-1}) = \begin{cases} 0, & c_r - c_{r-1} = 0, \\ P, & |c_r - c_{r-1}| = 1. \end{cases} \tag{3.42}$$

Hence, the optimal seam is the path $\{c_r\}_{r=0}^{H-1}$ that minimizes the sum of image-based energy and motion regularization. This formulation is particularly suitable for seam localization because it combines local appearance information with a global smoothness constraint.

During the forward recursion, a predecessor matrix is stored in parallel with the cumulative cost matrix. For each pixel (r, c), the predecessor matrix records the column index in row $r-1$ that yielded the minimum value in (3.39). Once the recursion reaches the last row, the endpoint of the optimal seam is found by

$$c^{\star}_{H-1} = \arg\min_{c} C(H-1, c). \tag{3.43}$$

The complete seam is then reconstructed by backtracking from the final row to the first row through the stored predecessor indices. This backtracking step converts the accumulated cost solution into an explicit pixel path.

The computational complexity of the forward DP stage is linear in the number of pixels, since each image location is visited once and only three predecessor candidates are evaluated. More precisely, the time complexity is $\mathcal{O}(HW)$, while the storage requirement is also $\mathcal{O}(HW)$ due to the cumulative cost and predecessor matrices. Although this complexity is already moderate, the repeated evaluation over all pixels can still become a bottleneck in real-time operation when high frame rates are required. For this reason, the implementation is accelerated using Numba just-in-time compilation, which compiles the nested loops into efficient machine code and significantly reduces the overhead associated with interpreted Python execution.

In addition to the standard forward recursion, the practical implementation includes a bidirectional refinement strategy. First, the optimal seam is computed in the original image orientation. Second, the same DP procedure is applied to the horizontally mirrored image, and the resulting seam indices are mapped back to the original coordinate system. For each row, the two candidate seam positions are compared and the darker pixel is selected. This bidirectional strategy reduces directional bias and improves robustness in the presence of local ambiguities or asymmetric texture responses. The final seam estimate is therefore obtained from a row-wise fusion of two DP-derived candidates rather than from a single forward pass only.

After the discrete seam path has been recovered, a polynomial curve $x = f(y)$ is

fitted to the extracted seam coordinates in order to suppress pixel-level irregularities and obtain a smoother geometric representation. Several polynomial degrees are evaluated, and the model with the lowest mean squared fitting error is selected. This post-processing stage is especially useful for downstream measurement and visualization, as it converts the raw discrete path into a continuous approximation suitable for estimating lateral deviation and displaying a stable seam centerline over the image.

Figure 3.7 illustrates the general principle of the dynamic programming seam search. Each node in the grid corresponds to a pixel state, and the optimal seam is obtained by recursively accumulating the minimum cost from one row to the next while storing predecessor links for backtracking.

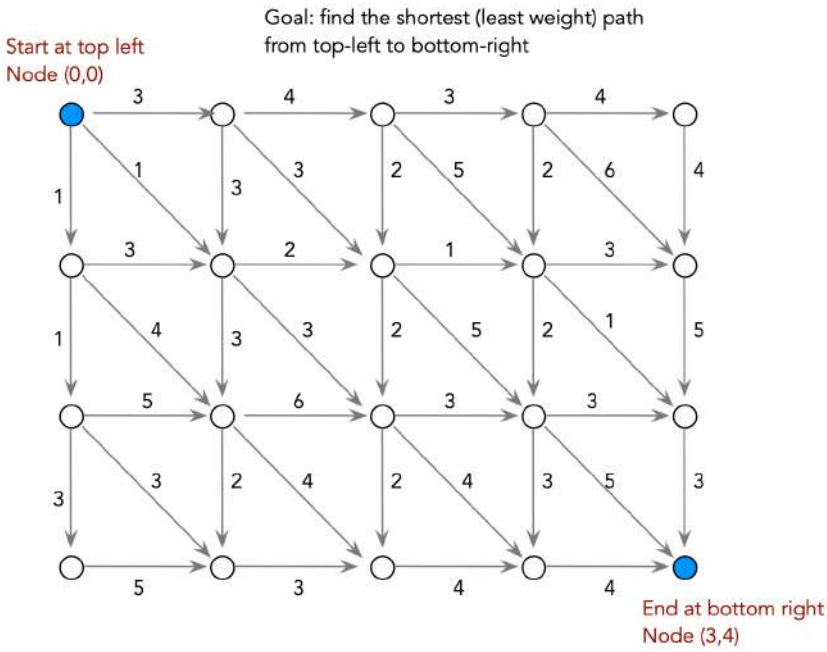


Figure 3.7.: Conceptual illustration of dynamic programming for seam extraction. The optimal seam is obtained by propagating cumulative costs row by row and then reconstructing the minimum-cost path by backtracking through the stored predecessors.

Overall, the dynamic programming formulation transforms seam localization from a purely local pixel selection problem into a globally constrained path optimization problem. By combining an energy map derived from Gabor filtering with a transition penalty that regularizes lateral motion, the method achieves reliable seam extraction even in the presence of noise, texture variation, and local contrast fluctuations. The use of predecessor tracking, bidirectional refinement, and just-in-time acceleration further improves both robustness and computational efficiency, making the approach well suited for real-time seam tracking in the proposed system.

Trajectory Regularization via Diagonal Transition Penalty

In the standard dynamic programming shortest-path formulation, an algorithm designed to strictly search for absolute minimum energy will behave in a purely greedy manner. In the context of fabric imagery, this presents a critical vulnerability: isolated dark spots—caused by sensor noise, lint, or micro-shadows—often generate lower local pixel intensity than the actual continuous seam. Without spatial regularization, the DP algorithm will erratically zig-zag across columns to "collect" these anomalously dark pixels, severely destabilizing the trajectory.

To counteract this, the system restricts the path's lateral degrees of freedom and introduces a spatial stiffness parameter known as the **Diagonal Transition Penalty** (P).

As defined in the optimal control formulation, from any given row r, the seam is only permitted to transition to three adjacent columns in the preceding row $r-1$: directly vertical ($\Delta c = 0$), diagonally left ($\Delta c = -1$), or diagonally right ($\Delta c = +1$) [KCTY23]. To mathematically discourage unnecessary lateral movement, an artificial cost scalar P is injected into the state transition function:

$$\text{Cost Transition} = \begin{cases} 0, & \text{if } \Delta c = 0 \text{ (Vertical)} \\ P, & \text{if } |\Delta c| = 1 \text{ (Diagonal)} \end{cases} \tag{3.44}$$

In this implementation, the penalty is empirically tuned to $P = 0.05$. Figure 3.8 visually deconstructs this mechanism. The penalty essentially acts as a low-pass mechanical filter for the mathematical path. The seam will prefer to travel in a straight, vertical downward trajectory (where the transition penalty is zero) unless a diagonal pixel is so significantly darker that it overcomes the $+0.05$ numerical resistance. This guarantees that the algorithm accurately tracks the macro-curvature of the physical seam while actively ignoring isolated, high-frequency noise spikes scattered across the fabric surface.

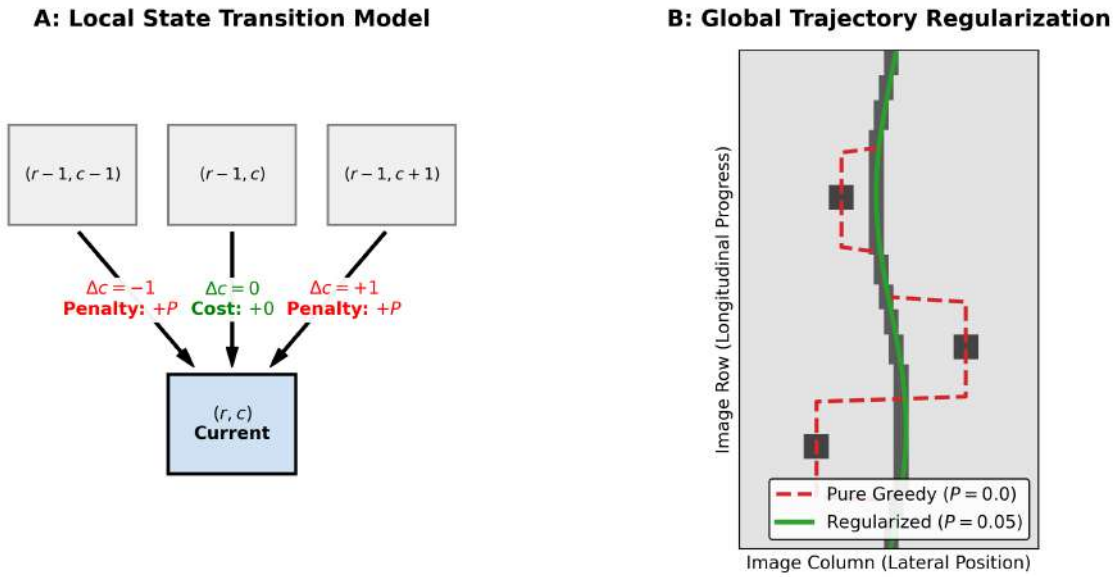


Figure 3.8.: Mechanism and effect of the diagonal transition penalty in the DP algorithm. (A) Local state transition model demonstrating the artificial cost $+P$ applied to lateral movements. (B) Global tracking simulation illustrating how a purely greedy algorithm ($P = 0.0$) gets derailed by dark isolated noise, whereas the regularized algorithm ($P = 0.05$) maintains structural continuity.

3.2.2. Bidirectional Seam Detection

A single seam search may introduce directional bias, especially in the presence of local ambiguities or asymmetric texture responses. To increase robustness, the seam detection is performed bidirectionally.

First, a seam is extracted from the original filtered image. Then, a second seam is extracted from the horizontally flipped image. After mapping the second result back into the original coordinate system, both candidates are fused row by row by selecting the darker pixel, i.e., the pixel with lower cost:

$$x_{\text{final}}(r) = \begin{cases} x_{\text{L2R}}(r), & \text{if } E(r, x_{\text{L2R}}(r)) < E(r, x_{\text{R2L}}(r)) \\ x_{\text{R2L}}(r), & \text{otherwise.} \end{cases} \tag{3.45}$$

This bidirectional strategy improves robustness against local artifacts and directional asymmetries, yielding a more stable seam estimate.

3.2.3. Polynomial Curve Fitting

The detected seam is represented as a discrete set of points

$$\{(x_i, y_i)\}_{i=1}^{N} \tag{3.46}$$

with one point per image row. Since the raw seam may still contain local irregularities, a polynomial model is fitted in order to obtain a smooth analytical representation suitable for visualization and geometric evaluation [BGH03, ZQZ11].

The seam is modeled as

$$x(y) = a_0 + a_1 y + a_2 y^2 + a_3 y^3 \tag{3.47}$$

where higher-order coefficients are omitted depending on the polynomial degree.

In the implementation, polynomial models of degree 1, 2, and 3 are evaluated. For each candidate model, the coefficients are obtained by least-squares fitting, and the quality of the fit is measured by the mean squared error (MSE):

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} \left(x_i - \hat{x}(y_i)\right)^2 . \tag{3.48}$$

The optimal polynomial degree is selected as

$$d^* = \arg \min_{d \in \{1,2,3\}} \mathrm{MSE}_d. \tag{3.49}$$

This fitting stage has two main advantages. First, it smooths the seam trajectory and reduces the effect of local pixel-level fluctuations. Second, it provides a continuous function $x = f(y)$ that can be evaluated at arbitrary row positions, which is useful for distance estimation and further control-oriented processing.

3.2.4. Distance Measurement

Once the seam is represented by a polynomial model, the system computes its horizontal deviation from a predefined vertical reference line. For a selected row position y_p, the seam coordinate is obtained as

$$x_{\mathrm{seam}}(y_p) = f(y_p). \tag{3.50}$$

The corresponding pixel distance relative to the reference line x_{ref} is

$$d_{\text{pix}}(y_p) = x_{\text{seam}}(y_p) - x_{\text{ref}}. \tag{3.51}$$

Using the calibration factor

$$s = 0.0125 \text{ mm/pixel}, \tag{3.52}$$

the distance is converted into millimeters as

$$d_{\text{mm}}(y_p) = d_{\text{pix}}(y_p) \cdot s. \tag{3.53}$$

In the implemented visualization, this distance is evaluated at multiple reference heights between two horizontal guide lines. This allows local inspection of seam deviation and provides direct metric information relevant for alignment analysis and quality monitoring.

3.2.5. Real-Time Implementation Using Threading

To achieve real-time performance, an alternative implementation is introduced in which image acquisition is decoupled from the image processing pipeline by means of threading. In general, threading is a programming technique that allows multiple tasks to make progress concurrently within the same process. Instead of executing all operations sequentially in a single control flow, the program is divided into multiple threads, each responsible for a specific task. This is particularly beneficial in real-time vision systems, where camera acquisition, image processing, visualization, and data logging may exhibit different timing characteristics.

In a single-threaded architecture, frame acquisition and processing are executed sequentially. As a result, the overall loop frequency is limited by the sum of the acquisition time and the processing time. If the camera read operation blocks, the processing stage remains idle while waiting for a new frame. Conversely, if image processing is computationally expensive, newly captured frames may be delayed or skipped, increasing end-to-end latency. Such behavior is undesirable in real-time applications, where timely response is often more important than processing every frame.

Let T_{acq} denote the average frame acquisition time and T_{proc} the average processing time. In a purely sequential implementation, the cycle time can be approximated as

$$T_{\text{seq}} = T_{\text{acq}} + T_{\text{proc}} \tag{3.54}$$

and the corresponding update rate is

$$f_{\text{seq}} = \frac{1}{T_{\text{seq}}}. \tag{3.55}$$

By contrast, when acquisition and processing are executed in separate threads, both tasks can progress quasi-independently. In this case, the effective cycle time is dominated by the slower stage:

$$T_{\text{thread}} \approx \max(T_{\text{acq}}, T_{\text{proc}}) \tag{3.56}$$

which yields an approximate update rate of

$$f_{\text{thread}} \approx \frac{1}{\max(T_{\text{acq}}, T_{\text{proc}})}. \tag{3.57}$$

Although this formulation is idealized and neglects synchronization overhead, it illustrates the main advantage of threaded execution: waiting time in one task does not necessarily stall the entire pipeline.

General Role of Threading in Real-Time Systems From a systems perspective, threading improves responsiveness by overlapping I/O-bound and compute-bound operations. Camera access is typically an I/O-bound task, since it depends on the delivery of new frames from hardware drivers or buffers. Image enhancement and seam localization, on the other hand, are compute-bound tasks, since they involve filtering, convolution, and cost-map evaluation. Separating these tasks into independent threads reduces idle time and improves temporal continuity.

Another important concept is latency. In real-time image processing, the goal is often not to process every acquired frame, but rather to process the most recent frame with minimal delay. Therefore, a threaded design is advantageous because it prioritizes recency over completeness. If the processing thread cannot keep pace with the acquisition rate, it can still operate on the latest available frame instead of accumulating an ever-growing queue of outdated frames.

Threaded Architecture in the Present Work To maximize throughput and prevent system bottlenecks, the real-time software architecture is partitioned into three distinct, asynchronously operating CPU threads. This strict separation ensures that physical hardware delays—such as camera exposure times or hard drive write speeds—do not stall the core computational pipeline. The task distribution is designed as follows:

- **Thread 1 (Camera Capture):** A dedicated I/O-bound thread responsible

solely for communicating with the XIMEA API and continuously grabbing the latest raw frame into a shared memory buffer.

- **Thread 2 (Image Saving):** A dedicated disk I/O thread that asynchronously handles the periodic writing of processed frames to the local storage, preventing file-system latency from freezing or delaying the live feed.
- **Thread 3 (Processing, Drawing, and UI):** The main compute-bound thread. It retrieves the latest available frame from Thread 1, executes the heavy image processing pipeline (CLAHE, Gaussian, Gabor, DP), overlays the measurement visuals, and refreshes the graphical user interface.

The shared frame buffer acts as the communication interface between these threads. Conceptually, the acquisition thread updates the buffer:

$$F_{\text{latest}}(t) \leftarrow \mathcal{A}(t) \tag{3.58}$$

where $\mathcal{A}(t)$ denotes the most recent frame delivered by the camera at time t, and F_{latest} represents the shared frame variable. Simultaneously, the processing thread (Thread 3) operates strictly on the current buffer content:

$$R(t) = \mathcal{P}\big(F_{\text{latest}}(t)\big) \tag{3.59}$$

where $\mathcal{P}(\cdot)$ denotes the seam localization pipeline and $R(t)$ the resulting seam estimate.

This architecture follows a *latest-frame strategy*: instead of enforcing strict one-to-one processing of all acquired frames (which can cause queuing delays), the system always processes the most recent available image. This is ideal for online seam tracking, since outdated frames have no practical value for closed-loop machine control.

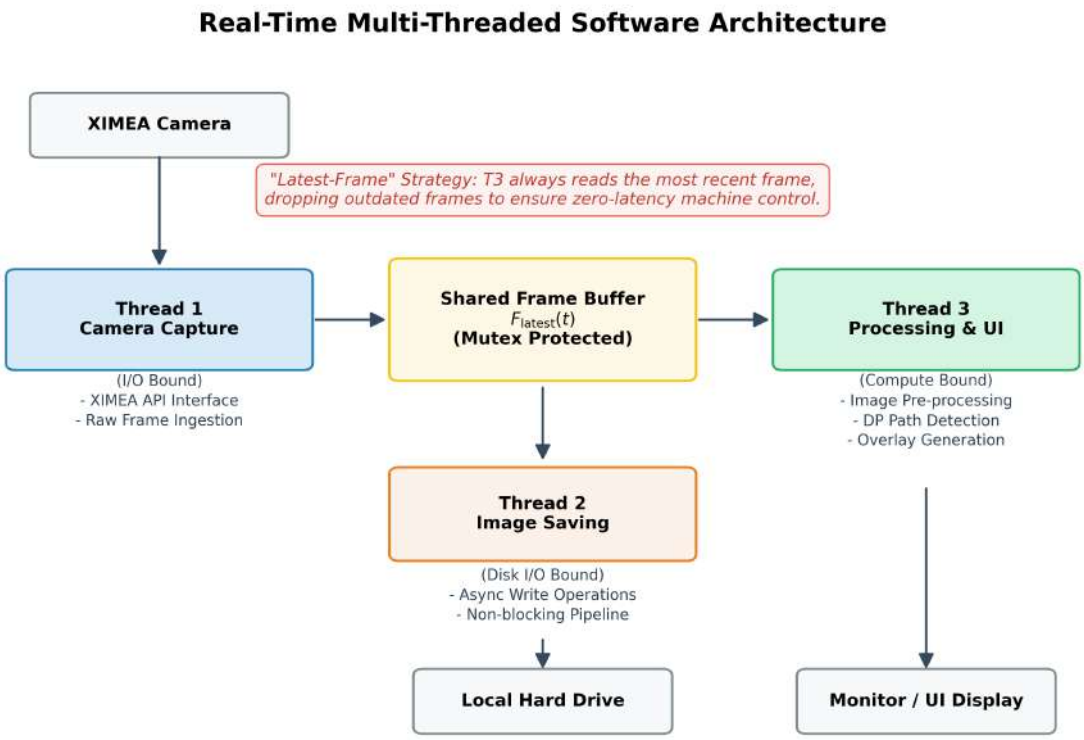


Figure 3.9.: The implemented multi-threaded software architecture. Decoupling the I/O-bound tasks (camera acquisition and disk saving) from the compute-bound image processing pipeline ensures that hardware latency does not disrupt the real-time continuous control of the sewing machine.

Synchronization and Shared Data Access Because both threads access shared data, synchronization must be considered carefully. In general, concurrent access to a shared image buffer can lead to race conditions, where one thread reads data while another thread is modifying it. To prevent inconsistent states, shared variables should be protected using synchronization primitives such as mutexes or locks. Let L denote a lock associated with the shared frame buffer. A safe access pattern can be abstractly written as

$$\text{lock}(L) \rightarrow \text{read/write shared frame} \rightarrow \text{unlock}(L). \tag{3.60}$$

This ensures that the processing thread always receives a valid frame and that the acquisition thread does not overwrite the buffer during read access. In practice, the synchronization overhead is small compared to the gain obtained by decoupling camera I/O from image processing.

Advantages for Seam Localization The use of threading provides several advantages in the proposed system. First, it reduces blocking behavior caused by synchronous camera operations. Second, it lowers the effective latency between frame capture and seam estimation. Third, it improves temporal smoothness by ensuring that the processing thread always works on recently acquired data. This is especially important in the present application, where the seam position must be estimated continuously and robustly under real-time constraints.

In addition, the threaded design improves scalability. As the image processing pipeline becomes more sophisticated through the use of CLAHE enhancement, Gaussian smoothing, Gabor filtering, and dynamic seam tracking, the computational load increases. A decoupled acquisition architecture makes it easier to maintain stable system behavior even when processing time varies slightly from frame to frame.

Practical Interpretation in the Present Project In the context of this project, threading is not introduced to perform parallel seam detection on multiple frames, but rather to separate two tasks with different temporal behavior: continuous camera acquisition and computational seam analysis. The acquisition thread continuously updates the most recent frame from the camera, while the main processing thread applies the enhancement and seam detection pipeline to that frame. As a result, the image processing stage is no longer forced to wait for each camera read operation, and the overall system becomes more responsive.

Therefore, the threaded implementation contributes directly to real-time seam path detection by reducing acquisition-induced delays and by allowing seam localization to operate on the most current visual information available.

3.2.6. Threaded Camera Acquisition

The threaded implementation is based on a dedicated camera class that encapsulates the XIMEA interface and continuously updates the latest frame in the background. Let I_t denote the most recent image available at time t. Instead of waiting for a new image inside the main processing loop, the algorithm directly accesses the latest buffered frame:

$$I_t = \text{CameraThread.read}(). \tag{3.61}$$

This producer-consumer-like structure improves responsiveness, since acquisition and processing can proceed asynchronously. As long as the camera thread updates frames at a sufficient rate, the main loop always operates on a recent image without stalling on image transfer.

3.2.7. Performance Considerations

To evaluate real-time performance, the processing loop measures the execution time per frame and computes the instantaneous frame rate as

$$\mathrm{FPS} = \frac{1}{\Delta t} \tag{3.62}$$

where Δt is the elapsed time required to process one frame.

The average frame rate is updated over a moving time interval and printed together with the measured seam displacements. The real-time capability of the system is achieved by combining three design choices:

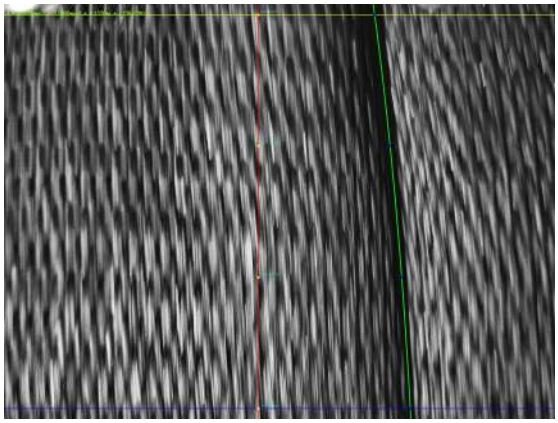
- threaded asynchronous acquisition,
- Numba-accelerated dynamic programming.

From an engineering perspective, this implementation represents a trade-off between geometric completeness and computational efficiency. The full seam-fitting approach provides richer geometric information, whereas the threaded two-point method prioritizes responsiveness and reduced latency. Together, both implementations demonstrate the flexibility of the proposed system for different monitoring and control scenarios.

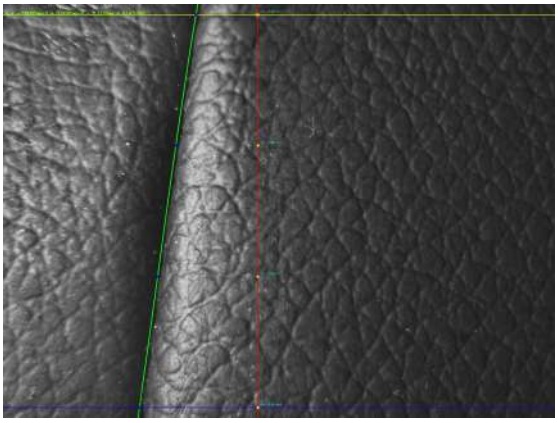
3.3. Result after System Integration

Following the individual implementation of the hardware acquisition and image processing components, the complete system was integrated and tested on the physical sewing machine setup. The proposed pipeline successfully extracts the optimal seam path and overlays the computational results directly onto the live fabric feed.

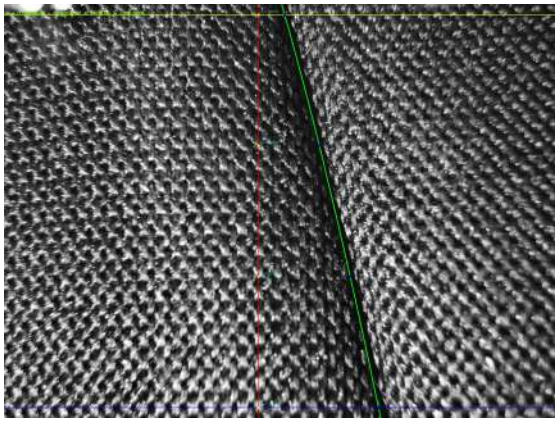
(a) Real-time camera view without any filter



(b) Tracking during high-speed movement.



(c) Detection on material variant 1.



(d) Detection on material variant 2.

Figure 3.10.: Results of the integrated real-time seam detection system, showing robust tracking across varying motion conditions and different material textures.

3.4. Software Output and Visualization

In addition to the visual overlay presented in the graphical user interface, the software is designed to operate as a continuous monitoring node. It outputs metric feedback to the console, allowing for headless operation (without a GUI) which is essential for direct integration with automated controllers or programmable logic controllers (PLCs).

The following terminal output demonstrates the real-time logging generated by the threaded Python script. The system continuously calculates the processing speed in Frames Per Second (FPS) alongside the spatial deviations (D_1 and D_2) in millimeters:

```
Command Prompt
C:\Users\Sara.Bavi\Desktop\Sara\04, threads_21-01.py
Opening camera...
System Ready.  High-Speed Mode Active.

FPS: 106.1 | Seam X: 1238, 1342 mm
FPS: 110.0 | Seam X: 1236, 1343 mm
FPS: 113.7 | Seam X: 1237, 1345 mm
FPS: 115.2 | Seam X: 1232, 1346 mm
FPS: 114.5 | Seam X: 1236, 1342 mm
FPS: 114.7 | Seam X: 1045, 2431 mm
FPS: 113.9 | Seam X: 1039, 2428 mm
FPS: 112.9 | Seam X: 1187, 1244 mm
FPS: 114.1 | Seam X: 1851, 2308 mm
FPS: 114.7 | Seam X: 1430, 1715 mm
FPS: 111.4 | Seam X: 2591, 2193 mm
FPS: 115.4 | Seam X: 1004, 1083 mm
FPS: 115.2 | Seam X:  510, 2226 mm
FPS: 115.2 | Seam X:  795, 2558 mm
FPS: 114.6 | Seam X:  581, 1989 mm
FPS: 114.0 | Seam X:  736, 2551 mm
```

Figure 3.11.: Command Prompt execution displaying the real-time processing frame rate and metric deviations.

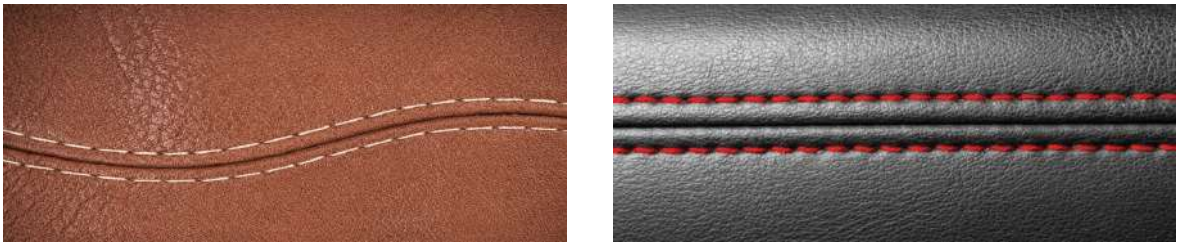
As observed in the log, the threaded algorithm stabilizes at an impressive rate of approximately 113 to 114 FPS. By converting pixel distances into metric values, the system provides a consistent and highly deterministic data stream of the fabric's physical deviation. This terminal logging validates that the vision system is fully capable of acting as a reliable, high-speed sensing component within a closed-loop control framework.

3.5. Final Automated Stitching Results

Following the successful integration of the vision system with the sewing machine's control unit, the fully automated setup was tested on physical fabric samples. The real-time trajectory data provided by the image processing pipeline enabled the machine to

dynamically adjust the needle position, actively compensating for fabric misalignment or manual feeding errors during the sewing process [KCTY23].

Figure 3.12 demonstrates the final physical output of the automated system. The decorative stitches are applied with high precision, maintaining a perfectly parallel and consistent distance from the detected central seam. The system successfully generalized across different materials, providing robust and uniform stitch placement on both high-contrast and low-contrast leather textures.



(a) Automated stitching on brown leather with curved seam

(b) Automated stitching on black leather with straight seam

Figure 3.12.: Final physical results of the automated decorative stitching process, demonstrating highly precise and parallel alignment relative to the main seam.

4. Verification and Validation

4.1. System Calibration, Path Control and Visualization

To verify the performance of the proposed system, the detected seam is visualized in real time by overlaying the estimated seam path on the live image stream. The seam is represented as a continuous curve obtained from polynomial fitting and displayed as a green trajectory, while a vertical red reference line represents the desired seam position.

This visual feedback allows qualitative assessment of detection accuracy and system stability under varying operating conditions.

4.1.1. Calibration and Metric Conversion

In order to convert image-based measurements into physically meaningful quantities, a pixel-to-millimeter calibration factor is introduced. Let the seam position at a given row y be defined as:

$$x_{\text{seam}}(y) = f(y) \tag{4.1}$$

where $f(y)$ is the fitted polynomial curve. The deviation from the reference line x_{ref} is:

$$d_{\text{pix}}(y) = x_{\text{seam}}(y) - x_{\text{ref}} \tag{4.2}$$

Using the calibrated scale factor:

$$s = 0.0125\,\text{mm/pixel} \tag{4.3}$$

the physical deviation is computed as:

$$d_{\text{mm}}(y) = d_{\text{pix}}(y) \cdot s \tag{4.4}$$

This transformation enables direct interpretation of seam displacement in real-world units, which is essential for industrial monitoring and control applications.

4.1.2. Multi-Point Deviation Evaluation

To improve robustness and spatial awareness, the system evaluates seam deviation at multiple predefined vertical positions: y_1, y_2, y_3, y_4.

At each point, the displacement is computed as:

$$d_i = (x_i - x_{\text{ref}}) \cdot s, \quad i = 1, \ldots, 4 \tag{4.5}$$

This multi-point evaluation provides a local approximation of seam curvature and allows detection of misalignment trends along the fabric.

4.1.3. Control-Oriented Interpretation

From a control systems perspective, the seam deviation $d(y)$ can be interpreted as an error signal:

$$e(y) = x_{\text{seam}}(y) - x_{\text{ref}} \tag{4.6}$$

The objective of the control system is to minimize this error:

$$\lim_{t \to \infty} e(y, t) \to 0 \tag{4.7}$$

The measured deviations can be used as feedback inputs to adjust:

- fabric position (via actuators),
- needle trajectory,
- or guiding mechanisms.

Thus, the vision system acts as a sensor layer within a closed-loop control system, providing real-time feedback for alignment correction.

4.1.4. Visualization and Real-Time Feedback

The system provides real-time visualization including:

- detected seam curve (green),
- reference trajectory (red),
- measurement points,
- numerical deviation values in millimeters.

Additionally, frame rate (FPS) is monitored to evaluate system performance and ensure real-time operation.

4.2. Challenges and Solutions

During the development of the system, several practical and computational challenges were encountered. These challenges arise from real-world conditions such as illumination variability, fabric texture complexity, mechanical disturbances, and computational constraints.

Table 4.1 summarizes the main challenges and the corresponding solutions implemented.

Table 4.1.: Summary of Challenges and Implemented Solutions

Challenge	**Solution**
Lighting noise and non-uniform illumination	Mitigated using CLAHE, which enhances local contrast while preventing over-amplification of noise. Additionally, controlled lighting conditions were used to stabilize brightness.
Variability in fabric textures and patterns	Addressed using a combination of CLAHE and Gabor filtering. The Gabor filter enhances oriented seam structures while suppressing irrelevant background textures.
Fabric wrinkles and geometric distortions	Reduced through bidirectional seam detection and polynomial curve fitting, which provide robustness against local irregularities and ensure smooth path estimation.
Mechanical vibrations (needle/fabric movement)	Mitigated by relying on frame-based processing stability and spatial smoothing via Gaussian filtering, reducing sensitivity to high-frequency disturbances.
High computational load	Reduced by using grayscale (MONO8) images, ROI-based processing in real-time mode, and Numba-accelerated dynamic programming.
Detection errors in curved or complex seam paths	Improved through polynomial model fitting (degree 1–3), allowing flexible representation of curved trajectories.
Low contrast between seam and fabric	Addressed through the combination of CLAHE and Gabor filtering, which enhances subtle intensity differences and improves seam separability.

4.2.1. Discussion of System Robustness

The combination of preprocessing techniques and optimization-based seam detection results in a system that is robust to a wide range of real-world disturbances. In particular:

- CLAHE ensures consistent contrast under varying lighting conditions.
- Gaussian filtering stabilizes the cost map.
- Gabor filtering enhances seam-oriented structures.
- Dynamic programming guarantees globally optimal path detection.
- Polynomial fitting smooths the trajectory and reduces local noise.

Furthermore, the introduction of bidirectional seam detection significantly improves reliability by reducing directional bias and increasing robustness to asymmetric artifacts.

4.2.2. Performance Trade-Offs

Two operational modes were developed to balance computational load and geometric completeness:

- **Full seam reconstruction:**
  - Provides complete geometric information.
  - Higher computational cost.
- **Two-point real-time detection (threaded):**
  - Faster execution.
  - Lower computational load.
  - Reduced geometric completeness.

This trade-off highlights the flexibility of the system, allowing it to be adapted depending on application requirements such as accuracy versus speed.

5. Conclusion and Future Work

5.1. Conclusion

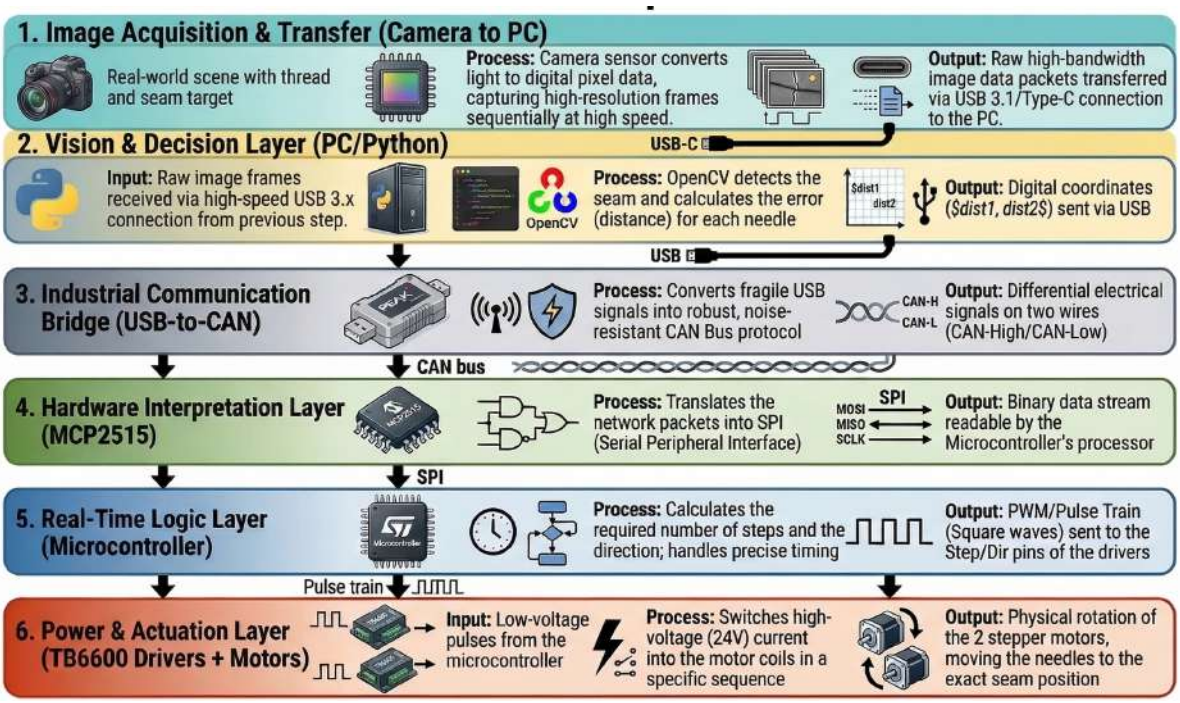


Figure 5.1.: Visual summary of the intelligent real-time seam tracking concept.

This work presented the design and implementation of a real-time vision-based system...

This work presented the design and implementation of a real-time vision-based system for seam path detection and measurement in textile manufacturing. The system integrates image processing, optimization algorithms, and real-time computation to provide accurate and robust seam localization under varying operating conditions.

The proposed pipeline combines CLAHE-based contrast enhancement, Gaussian smoothing, and Gabor filtering to transform raw grayscale images into a representation where seam structures are clearly distinguishable. Based on this processed image, a dynamic programming approach is used to extract the seam as a globally optimal path, while a bidirectional evaluation strategy improves robustness against noise and

directional bias. The resulting seam is further refined through polynomial curve fitting, enabling smooth and continuous representation suitable for geometric analysis.

In addition to full seam reconstruction, a computationally efficient real-time variant was developed using threaded image acquisition and region-of-interest (ROI) processing. This implementation allows fast estimation of seam position at selected points, demonstrating the system's capability to operate under real-time constraints.

A key contribution of this work is the integration of image-based seam detection with metric distance estimation. By converting pixel measurements into millimeters through calibration, the system provides physically meaningful feedback that can be directly used for process monitoring and control. From a systems perspective, the seam deviation can be interpreted as an error signal, enabling the vision module to function as a sensing component within a closed-loop control framework.

Overall, the developed system demonstrates that combining computer vision techniques with optimization-based path detection and real-time processing leads to a robust and flexible solution for seam tracking. The results indicate a clear improvement in precision, consistency, and repeatability compared to manual alignment, thereby contributing to increased efficiency and quality in textile manufacturing processes.

5.2. Future Work

Although the proposed system achieves reliable seam detection and real-time performance, several directions can be explored to further enhance its capabilities and move toward industrial deployment.

5.2.1. Closed-Loop Control Integration

The most important extension of this work is the integration of the vision system into a fully closed-loop control architecture. In this scenario, the measured seam deviation would be used as feedback to actively control the position of the fabric or the needle in real time. This would enable automatic correction of misalignment during stitching and significantly improve process accuracy.

Future work should focus on designing and implementing suitable control strategies, such as proportional-integral-derivative (PID) control or model-based control approaches, to regulate the seam position based on the measured error signal.

5.2.2. Robustness Enhancement and Noise Compensation

Although the current preprocessing pipeline already provides robustness against noise and illumination variations, additional improvements can be achieved by incorporating more advanced filtering and estimation techniques to implement further noise compensation. Possible extensions include:

- temporal filtering across consecutive frames to reduce measurement noise,
- adaptive parameter tuning based on image conditions,
- and more advanced texture or feature-based representations.

Such improvements would increase system stability under challenging industrial conditions, including varying lighting, complex fabric textures, and dynamic disturbances.

5.2.3. Algorithm Optimization for Industrial Deployment

To ensure safe and reliable operation in real production environments, further optimization of the algorithm is required. This includes:

- reducing computational latency,
- improving memory efficiency,
- and ensuring deterministic execution times.

Special attention should be given to real-time constraints, particularly in high-speed sewing processes where rapid response is essential.

5.2.4. Hardware Acceleration and Embedded Deployment

Another important direction is the transition from a desktop-based implementation to an embedded or edge-computing platform. Potential hardware platforms include:

- NVIDIA Jetson devices for GPU-accelerated processing,
- or lightweight systems, low-cost deployment.

This step requires adaptation of the software architecture, optimization of computational resources, and validation of real-time performance on embedded hardware.

5.2.5. Extended Experimental Validation

Finally, further validation is required through extensive real-world testing under different operating conditions. This includes:

- evaluating performance under varying lighting conditions,
- and analyzing long-term stability during continuous operation.

Such experiments would provide deeper insight into system robustness and reliability, and support the transition from prototype to industrial application.

A. Full Seam Detection Script

The following appendix contains the complete Python implementation for the real-time bidirectional seam path detection system. This script handles the XIMEA camera initialization, image preprocessing (CLAHE, Gaussian Blur, Gabor filtering), dynamic programming-based seam extraction, polynomial curve fitting, and the real-time visualization with metric distance calculation.

```
import sys
import os
import time
from datetime import datetime
import cv2
import numpy as np
from numba import njit

# --------------------
# XIMEA setup
# --------------------
sys.path.append(r"C:/Users/Sara.Bavi/Desktop/Sara/Ximea_MP9PU/MY
    python/Python/v3/ximea")
import xiapi

SAVE_DIR = r"C:\Users\Sara.Bavi\Desktop\Sara\Ximea_MP9PU\MY python\
    save images1\real images"
OUTPUT_DIR = r"C:\Users\Sara.Bavi\Desktop\Sara\Ximea_MP9PU\MY python\
    save images1\real images\merged"
os.makedirs(SAVE_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)

# --------------------
# Config
# --------------------
GAUSS_KSIZE = 9
DIAG_PENALTY = 0.05
POLY_DEGREES = (1, 2, 3)
CLAHE_CLIP = 2.0
CLAHE_TILE = 8
PIXEL_SIZE_MM = 0.0125
```

```python

# --------------------
# Seam finder
# --------------------
@njit
def find_vertical_dark_seam_numba(gray, diag_penalty=0.05):
    h, w = gray.shape
    energy = gray.astype(np.float64)
    cost = np.full((h, w), np.inf, dtype=np.float64)
    prev = np.full((h, w), -1, dtype=np.int32)
    cost[0, :] = energy[0, :]

    for r in range(1, h):
        for c in range(w):
            best_val = np.inf
            best_c = -1
            for dc in (-1, 0, 1):
                pc = c + dc
                if 0 <= pc < w:
                    pen = diag_penalty if dc != 0 else 0.0
                    v = cost[r-1, pc] + energy[r, c] + pen
                    if v < best_val:
                        best_val, best_c = v, pc
            cost[r, c] = best_val
            prev[r, c] = best_c

    end_c = np.argmin(cost[-1, :])
    x_path = np.empty(h, dtype=np.int32)
    y_path = np.empty(h, dtype=np.int32)
    c = end_c
    for r in range(h-1, -1, -1):
        if c < 0:
            c = x_path[r+1] if r < h-1 else 0
        x_path[r] = c
        y_path[r] = r
        c = prev[r, c] if c >= 0 else -1
    return x_path, y_path

# --------------------
# Bidirectional seam detection
# --------------------
def find_vertical_dark_seam_bidirectional(gray, diag_penalty=
    DIAG_PENALTY):
    # Left-to-right
    x_l2r, y = find_vertical_dark_seam_numba(gray, diag_penalty)
```

```
    # Right-to-left
    x_r2l, _ = find_vertical_dark_seam_numba(gray[:, ::-1],
   diag_penalty)
    x_r2l = gray.shape[1] - 1 - x_r2l  # reverse indices

    # Merge: choose darkest pixel for each row
    row_indices = np.arange(gray.shape[0])
    vals_l2r = gray[row_indices, x_l2r]
    vals_r2l = gray[row_indices, x_r2l]
    x_final = np.where(vals_l2r < vals_r2l, x_l2r, x_r2l)
    return x_final, y

# --------------------
# Polynomial fit
# --------------------
def best_polyfit_x_of_y(x, y, degrees=(1, 2, 3)):
    best = {'degree': None, 'coeffs': None, 'mse': np.inf}
    for d in degrees:
        coeffs = np.polyfit(y, x, d)
        pred = np.poly1d(coeffs)(y)
        mse = np.mean((x - pred) ** 2)
        if mse < best['mse']:
            best = {'degree': d, 'coeffs': coeffs, 'mse': mse}
    return best

def equation_text_from_coeffs(coeffs, tol=1e-10, digits=4):
    terms = []
    degree = len(coeffs) - 1
    for i, a in enumerate(coeffs):
        power = degree - i
        if abs(a) < tol:
            continue
        if power == 0:
            terms.append(f"{a:.{digits}f}")
        elif power == 1:
            terms.append(f"{a:.{digits}f}*y")
        else:
            terms.append(f"{a:.{digits}f}*y^{power}")
    if not terms:
        return "x = 0"
    return "x = " + " + ".join(terms).replace("+ -", "- ")

# --------------------
# Camera Init
# --------------------
```

```
cam = xiapi.Camera()
print("Opening camera...")
cam.open_device()
cam.set_imgdataformat('XI_MONO8')
exposure_time = 10000
cam.set_exposure(exposure_time)
cam.start_acquisition()
print("Capturing images... Press 'X' to stop.")

img = xiapi.Image()
clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP, tileGridSize=(CLAHE_TILE
    , CLAHE_TILE))
last_save_time = time.time()

# --------------------
# Gabor filter parameter
# --------------------
def apply_gabor(gray):
    ksize = 31
    sigma = 7.0
    theta = np.pi/2 - 0.25
    lamda = 10.0
    gamma = 0.5
    psi = 0
    gabor_kernel = cv2.getGaborKernel((ksize, ksize), sigma, theta,
    lamda, gamma, psi, ktype=cv2.CV_32F)
    filtered = cv2.filter2D(gray, cv2.CV_8UC1, gabor_kernel)
    return filtered

# --------------------
# Main loop
# --------------------
while True:
    cam.get_image(img)
    frame = img.get_image_data_numpy()
    if frame is None:
        continue

    frame = cv2.rotate(frame, cv2.ROTATE_180)
    H_orig, W_orig = frame.shape

    # -------------------- Image processing --------------------
    frame_clahe = clahe.apply(frame)
    img_blur = cv2.GaussianBlur(frame_clahe, (GAUSS_KSIZE, GAUSS_KSIZE
    ), 0)
```

```
    img_gabor = apply_gabor(img_blur)

    # *** Bidirectional seam detection ***
    x_path, y_path = find_vertical_dark_seam_bidirectional(img_gabor, DIAG_PENALTY)
    fit = best_polyfit_x_of_y(x_path.astype(np.float64), y_path.astype(np.float64), POLY_DEGREES)
    eq_text = equation_text_from_coeffs(fit['coeffs'])

    y_vals = np.linspace(0, H_orig - 1, H_orig)
    x_vals = np.poly1d(fit['coeffs'])(y_vals)
    pts = np.column_stack([x_vals, y_vals]).astype(np.int32)

    img_color = cv2.cvtColor(frame_clahe, cv2.COLOR_GRAY2BGR)

    # Green curve
    for i in range(len(pts) - 1):
        x1, y1 = pts[i]
        x2, y2 = pts[i + 1]
        if 0 <= x1 < W_orig and 0 <= x2 < W_orig:
            cv2.line(img_color, (x1, y1), (x2, y2), (0, 255, 0), 3)

    # Reference lines
    red_x = (W_orig // 2) - 100
    cv2.line(img_color, (red_x, 0), (red_x, H_orig), (0, 0, 255), 2)

    blue_y = H_orig -50
    yellow_y = 50
    cv2.line(img_color, (0, blue_y), (W_orig, blue_y), (255, 0, 0), 2)
    cv2.line(img_color, (0, yellow_y), (W_orig, yellow_y), (0, 255, 255), 2)

    # 4 Ref. points
    num_points = 4
    y_top = min(blue_y, yellow_y)
    y_bottom = max(blue_y, yellow_y)
    y_points = np.linspace(y_top, y_bottom, num_points).astype(int)

    for yp in y_points:
        cv2.circle(img_color, (red_x, yp), 6, (0, 255, 255), -1)
        curve_x = int(np.poly1d(fit['coeffs'])(yp))
        cv2.circle(img_color, (curve_x, yp), 6, (255, 0, 0), -1)
        dist_pixel = curve_x - red_x
        dist_mm = dist_pixel * PIXEL_SIZE_MM
```

```
        cv2.putText(img_color, f"d={dist_mm:.2f}mm", (red_x+10, yp-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

    cv2.putText(img_color, eq_text, (10, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 128), 2)

    display = cv2.resize(img_color, (W_orig // 2, H_orig // 2))
    cv2.imshow("Seam Detection", display)

    # Save periodically
    if time.time() - last_save_time >= 0.2:
        filename = os.path.join(OUTPUT_DIR, datetime.now().strftime("%Y%m%d_%H%M%S") + ".jpg")
        cv2.imwrite(filename, img_color)
        print(f"Saved: {filename}")
        last_save_time = time.time()

    # Control exposure and stop camera
    key = cv2.waitKey(1) & 0xFF
    if key in [ord('x'), ord('X')]:
        break
    elif key == ord('+'):
        exposure_time += 1000
        cam.set_exposure(exposure_time)
        print(f"Exposure increased: {exposure_time}")
    elif key == ord('-'):
        exposure_time = max(0, exposure_time - 1000)
        cam.set_exposure(exposure_time)
        print(f"Exposure decreased: {exposure_time}")

cam.stop_acquisition()
cam.close_device()
cv2.destroyAllWindows()
```

Listing A.1: Bidirectional Seam Finding Script

B. Real-Time Threaded Full Seam Detection Script

The following appendix contains the advanced Python implementation utilizing a multi-threaded architecture for high-performance seam tracking. This version decouples image acquisition, processing, and disk I/O into separate threads, significantly improving frame rates and system responsiveness. It also includes automated background image saving via a queue-based system and real-time exposure control.

```
import sys
import os
import time
import threading
import queue
from datetime import datetime
import cv2
import numpy as np
from numba import njit

# --------------------
# XIMEA setup
# --------------------
sys.path.append(r"C:/Users/Sara.Bavi/Desktop/Sara/Ximea_MP9PU/MY
    python/Python/v3/ximea")
import xiapi

SAVE_DIR = r"C:\Users\Sara.Bavi\Desktop\Sara\Ximea_MP9PU\MY python\
    save images1\real images"
OUTPUT_DIR = r"C:\Users\Sara.Bavi\Desktop\Sara\Ximea_MP9PU\MY python\
    save images1\real images\merged"
os.makedirs(SAVE_DIR, exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)

# --------------------
# Config
# --------------------
GAUSS_KSIZE = 9
```

```
DIAG_PENALTY = 0.05
POLY_DEGREES = (1, 2, 3)
CLAHE_CLIP = 2.0
CLAHE_TILE = 6
PIXEL_SIZE_MM = 0.0125
RESIZE_FACTOR = 0.4

# --------------------
# 1. Threaded Camera Class
# --------------------
class XimeaThreadedCamera:
    def __init__(self, exposure=10000):
        self.cam = xiapi.Camera()
        print("Opening camera...")
        self.cam.open_device()
        self.cam.set_imgdataformat('XI_MONO8')
        self.cam.set_exposure(exposure)
        self.cam.start_acquisition()

        self.img = xiapi.Image()
        self.frame = None
        self.stopped = False
        self.exposure = exposure

        self.thread = threading.Thread(target=self.update, daemon=True
    )
        self.thread.start()

    def update(self):
        while not self.stopped:
            try:
                self.cam.get_image(self.img)
                raw = self.img.get_image_data_numpy()
                self.frame = cv2.rotate(raw, cv2.ROTATE_180)
            except Exception as e:
                print(f"Camera Error: {e}")
                continue

    def read(self):
        return self.frame

    def set_exposure(self, val):
        self.exposure = max(0, val)
        self.cam.set_exposure(self.exposure)
        print(f"Exposure set to: {self.exposure}")
```

```python

    def stop(self):
        self.stopped = True
        self.cam.stop_acquisition()
        self.cam.close_device()

# --------------------
# 2. Threaded Image Saver Class
# --------------------
class ImageSaverThread:
    def __init__(self):
        self.queue = queue.Queue(maxsize=30)
        self.stopped = False
        self.thread = threading.Thread(target=self.worker, daemon=True
    )
        self.thread.start()

    def worker(self):
        while not self.stopped or not self.queue.empty():
            try:
                frame, filename = self.queue.get(timeout=1)
                cv2.imwrite(filename, frame)
                self.queue.task_done()
            except queue.Empty:
                continue

    def save(self, frame, filename):
        if not self.queue.full():
            self.queue.put((frame.copy(), filename))

    def stop(self):
        self.stopped = True
        self.thread.join()

# --------------------
# 3. Processing Functions
# --------------------
@njit
def find_vertical_dark_seam_numba(gray, diag_penalty=0.05):
    h, w = gray.shape
    energy = gray.astype(np.float64)
    cost = np.full((h, w), np.inf, dtype=np.float64)
    prev = np.full((h, w), -1, dtype=np.int32)
    cost[0, :] = energy[0, :]
    for r in range(1, h):
```

```
        for c in range(w):
            best_val = np.inf
            best_c = -1
            for dc in (-1, 0, 1):
                pc = c + dc
                if 0 <= pc < w:
                    pen = diag_penalty if dc != 0 else 0.0
                    v = cost[r-1, pc] + energy[r, c] + pen
                    if v < best_val:
                        best_val, best_c = v, pc
            cost[r, c] = best_val
            prev[r, c] = best_c
    end_c = np.argmin(cost[-1, :])
    x_path = np.empty(h, dtype=np.int32)
    c = end_c
    for r in range(h-1, -1, -1):
        x_path[r] = c
        c = prev[r, c] if c >= 0 else 0
    return x_path

def best_polyfit_x_of_y(x, y, degrees=(1, 2, 3)):
    best = {'degree': None, 'coeffs': None, 'mse': np.inf}
    for d in degrees:
        coeffs = np.polyfit(y, x, d)
        pred = np.poly1d(coeffs)(y)
        mse = np.mean((x - pred) ** 2)
        if mse < best['mse']:
            best = {'degree': d, 'coeffs': coeffs, 'mse': mse}
    return best

def apply_gabor(gray):
    ksize, sigma, theta, lamda, gamma = 31, 7.0, np.pi/2 - 0.25, 10.0, 0.5
    gabor_kernel = cv2.getGaborKernel((ksize, ksize), sigma, theta, lamda, gamma, 0, ktype=cv2.CV_32F)
    return cv2.filter2D(gray, cv2.CV_8UC1, gabor_kernel)

# --------------------
# 4. Main Execution
# --------------------
camera = XimeaThreadedCamera(exposure=10000)
saver = ImageSaverThread()
clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP, tileGridSize=(CLAHE_TILE, CLAHE_TILE))

```

```
fps_list = []
fps_timer = time.time()
last_save_time = time.time()

print("System Ready. Press 'X' to exit.")

try:
    while True:
        frame_full = camera.read()
        if frame_full is None:
            continue

        t_start = time.time()
        H_full, W_full = frame_full.shape

        H_proc, W_proc = int(H_full*RESIZE_FACTOR), int(W_full*RESIZE_FACTOR)
        frame_small = cv2.resize(frame_full, (W_proc, H_proc))

        frame_clahe = clahe.apply(frame_small)
        img_blur = cv2.GaussianBlur(frame_clahe, (GAUSS_KSIZE, GAUSS_KSIZE), 0)
        img_gabor = apply_gabor(img_blur)

        y_path_small = np.arange(H_proc)
        x_path_small = find_vertical_dark_seam_numba(img_gabor, DIAG_PENALTY)
        fit = best_polyfit_x_of_y(x_path_small.astype(np.float64), y_path_small.astype(np.float64), POLY_DEGREES)

        y_vals_full = np.linspace(0, H_full - 1, H_full)
        coeffs_full = np.polyfit(y_path_small/RESIZE_FACTOR, x_path_small/RESIZE_FACTOR, fit['degree'])
        poly_full = np.poly1d(coeffs_full)

        img_color = cv2.cvtColor(frame_full, cv2.COLOR_GRAY2BGR)
        red_x = (W_full // 2) - 100

        y_points = np.linspace(50, H_full - 50, 4).astype(int)
        for yp in y_points:
            cur_x = int(poly_full(yp))
            cv2.circle(img_color, (cur_x, yp), 6, (0, 255, 255), -1)
            dist_mm = (cur_x - red_x) * PIXEL_SIZE_MM
            cv2.putText(img_color, f"{dist_mm:.2f}mm", (red_x+10, yp-10), 0, 0.5, (255, 255, 0), 1)
```

```python

        if time.time() - last_save_time >= 0.2:
            fname = os.path.join(OUTPUT_DIR, datetime.now().strftime("%H%M%S_%f") + ".jpg")
            saver.save(img_color, fname)
            last_save_time = time.time()

        fps = 1.0 / (time.time() - t_start)
        fps_list.append(fps)
        if time.time() - fps_timer >= 2.0:
            print(f"FPS: {sum(fps_list)/len(fps_list):.1f} | Queue: {saver.queue.qsize()}")
            fps_list, fps_timer = [], time.time()

        display = cv2.resize(img_color, (W_full // 2, H_full // 2))
        cv2.imshow("Seam Detection", display)

        key = cv2.waitKey(1) & 0xFF
        if key == ord('x'): break
        elif key == ord('+'): camera.set_exposure(camera.exposure + 1000)
        elif key == ord('-'): camera.set_exposure(camera.exposure - 1000)

finally:
    camera.stop()
    saver.stop()
    cv2.destroyAllWindows()
```

Listing B.1: Threaded Camera and Image Saver Script

Bibliography

[BF07] Francesco Bianconi and Antonio Fernández. Evaluation of the effects of gabor filter parameters on texture classification. *Pattern Recognition*, 2007.

[BGH03] Soumen Biswas, Dibyendu Ghoshal, and Ranjay Hazra. A new algorithm of image segmentation using curve fitting based higher order polynomial smoothing. 2003. Dream Institute of Technology, National Institute of Technology Agartala, IIT Roorkee.

[CCP05] Che-Seung Cho, Byeong-Mook Chung, and Moo-Jin Park. Development of real-time vision-based fabric inspection system. *IEEE Transactions on Industrial Electronics*, 52(4), 2005.

[CZGL20] Ling Chen, Shan Zeng, Quansheng Gao, and Biao Liu. Adaptive gabor filtering for fabric defect inspection. *Journal of Computers*, 31(2):45–55, 2020.

[DC20] G. Deng and L. W. Cahill. An adaptive gaussian filter for noise reduction and edge detection. 2020. Department of Electronic Engineering, La Trobe University, Australia.

[DH95] Dennis Dunn and William E. Higgins. Optimal gabor filters for texture segmentation. *IEEE Transactions on Image Processing*, 4(7), 1995.

[DLPK15] Virodhi Dakshayani, Govinda Rao Locharla, Paweł Pławiak, and Chiranjeevi Karri. Design of a gabor filter-based image denoising hardware model. *MDPI*, 2015.

[ED24] Mehmet Erdogan and Mustafa Dogan. Enhanced curvature-based fabric defect detection: A experimental study with gabor transform and deep learning. *Algorithms*, 2024.

[Get13] Pascal Getreuer. A survey of gaussian convolution algorithms. *Image Processing On Line*, 2013.

[GPK02] Simona E. Grigorescu, Nicolai Petkov, and Peter Kruizinga. Comparison of texture features based on gabor filters. *IEEE Transactions on Image Processing*, 11(10), 2002.

[JRL96] Anil K. Jain, Nalini K. Ratha, and Sridhar Lakshmanan. Object detection using gabor filters. *Pattern Recognition*, 1996.

[JYLK14] Junfeng Jing, Panpan Yang, Pengfei Li, and Xuejuan Kang. Supervised defect detection on textile fabrics via optimal gabor filter. *Journal of Industrial Textiles*, 44(1):40–57, 2014.

[Kam14] Joni-Kristian Kamarainen. Gabor features in image analysis. *Image Processing Theory, Tools and Applications*, 2014.

[KCTY23] Naoki Kosaka, Yuichi Chida, Masaya Tanemura, and Kimitoshi Yamazaki. Real-time optimal control of automatic sewing considering fabric geometric shapes. *Mechatronics*, 2023.

[KWB22] Chung-Feng Jeffrey Kuo, Wei-Ren Wang, and Jagadish Barman. Automated optical inspection for defect identification and classification in actual woven fabric production lines. *Sensors*, 2022.

[KYMP05] Amandeep Kour, Vimal Kishore Yadav, Vikas Maheshwari, and Deepak Prashar. A review on image processing. *International Journal of Electronics Communication and Computer Engineering*, 4(1), 2005.

[LLL+21] Chao Li, Jun Li, Yafei Li, Lingmin He, Xiaokang Fu, and Jingjing Chen. Fabric defect detection in textile manufacturing: A survey of the state of the art. *Security and Communication Networks*, 2021, 2021.

[MKP09] P. M. Mahajan, S. R. Kolhe, and P. M. Patil. A review of automatic fabric defect detection techniques. *Advances in Computational Research*, 1(2):18–29, 2009.

[PSS20] Priyanka, Rishabh Shukla, and Laxmi Shrivastava. Image restoration of image with gaussian filter. *International Research Journal of Engineering and Technology (IRJET)*, 7(12), 2020.

[PVW17] Minal Patil, Sarita Verma, and Jaya Wakode. A review on fabric defect detection techniques. *International Research Journal of Engineering and Technology (IRJET)*, 4(9), 2017.

[RZR+20] Aqsa Rasheed, Bushra Zafar, Amina Rasheed, Nouman Ali, Saadat Hanif Dar, Usman Habib, Tehmina Shehryar, and Muhammad Tariq Mahmood. Fabric defect detection using computer vision techniques: A comprehensive review. *Mathematical Problems in Engineering*, 2020, 2020.

[Sha23] Richa Sharma. A review on clahe based enhancement techniques. In *2023 6th International Conference on Contemporary Computing and Informatics (IC3I)*. IEEE, 2023.

[SPYJ05] Dong-Hyuk Shin, Rae-Hong Park, Seungjoon Yang, and Jae-Han Jung. Block-based noise estimation using adaptive gaussian filtering. *IEEE Transactions on Consumer Electronics*, 51(1), 2005.

[VJDBSS22] S. Venkatesh, C. John De Britto, P. Subhashini, and K. Somasundaram. Image enhancement and implementation of clahe algorithm and bilinear interpolation. *Cybernetics and Systems*, 2022.

[XIM17] XIMEA GmbH. *xiMU Subminiature USB 2.0 Cameras Brochure*, 2017. Technical Datasheet/Brochure.

[Yu23] Jingning Yu. Based on gaussian filter to improve the effect of the images in gaussian noise and pepper noise. *Journal of Physics: Conference Series*, 2580, 2023.

[ZQZ11] Liang Zhang, Yongyuan Qin, and Jinliang Zhang. Study of polynomial curve fitting algorithm for outlier elimination. In *2011 International Conference on Computer Science and Service System (CSSS)*. IEEE, 2011.

[ZXW13] Jie Zhang, Binjie Xin, and Xiangji Wu. A review of fabric identification based on image analysis technology. *Textiles and Light Industrial Science and Technology (TLIST)*, 2(3), 2013.